Gestión de asientos en el aula por día

Caso práctico · Unidad 2 · Estructuras de datos básicas

Estructura de datos lineal implementada: COLA (FIFO)

Carlos de Jesús Martínez Pineda · 253432
Carlos Adrián Alarcón Valencia · 253580

4° C · Estructuras de Datos · 4 de noviembre de 2026

Datos generales

Nombres de los estudiantes	Carlos de Jesús Martínez Pineda Carlos Adrián Alarcón Valencia
Matrículas	253432 253580
Asignatura	Estructuras de Datos
Nombre de la unidad	Estructuras de datos básicas
Fecha de entrega	4 de noviembre de 2026
Caso asignado	Caso 7. Gestión de asientos en el aula, por día, considerar 2 días. El alumno decide en qué asiento utilizar.
Estructura lineal	Cola (Queue) — principio FIFO

Enunciado

> *«Gestión de asientos en el aula, por día, considerar 2 días. El alumno decide en qué asiento utilizar.»*

El caso práctico pide administrar los asientos de un aula durante dos jornadas de clase. En cada jornada el docente pasa lista: los alumnos se atienden conforme llegan, y cada alumno elige la fila y la silla que va a ocupar, sin que dos alumnos puedan quedar en el mismo lugar. Al terminar la jornada se requiere conocer cuántos asistieron, cuántos faltaron y qué asiento ocupó cada quien; al terminar los dos días se requiere comparar ambas jornadas.

De la lectura del enunciado se desprenden cuatro necesidades concretas:

- **Turno:** respetar el orden de llegada de los alumnos (un turno),
- **Elección de asiento:** permitir que el alumno elija fila y silla, y evitar asientos duplicados.
- **Registro por día:** guardar el resultado de cada día por separado, considerando exactamente dos días.
- **Reporte:** obtener totales de asistencia y faltas por jornada y una comparación entre ambas.

Etapas de resolución de problemas

1. Análisis y diseño del problema

Justificación

1. Identificación del principio de acceso. La lógica del caso es **FIFO (First In, First Out)**: el primer alumno que llega es el primero que elige asiento. El orden de llegada es parte del problema (es un pase de lista), no un detalle de implementación; por eso la estructura lineal adecuada es una **cola**.

2. Relación estructura-caso. Las operaciones de la cola modelan directamente el flujo del pase de lista:

- **Encolar:** cada alumno de la lista entra al final de la cola con offer();
- **Consultar:** la tarjeta de turno consulta al frente con peek(), sin retirarlo;
- **Desencolar:** al elegir silla (o marcarse ausente) el alumno sale con poll(), y solo entonces se ocupa la posición en la matriz de asientos;
- **Cierre:** los alumnos que quedan en la cola al cerrar el día se cuentan como faltas.

La cola se declara como Queue<Alumno> y se instancia con LinkedList. Aunque LinkedList también implementa List, el programa la utiliza a través de la interfaz Queue, de modo que la única forma de sacar elementos es por el frente: la estructura impide, por diseño, atender a un alumno que no es su turno.

3. Descarte de alternativas. Las otras dos estructuras lineales no satisfacen el caso:

- **Pila:** atendería al último alumno que llegó (LIFO), es decir, invertiría el orden del pase de lista. Sería útil si el caso pidiera deshacer la última acción, pero aquí el orden de llegada es el requisito central.
- **Lista:** permitiría insertar, consultar o eliminar a cualquier alumno en cualquier posición. Eso resuelve el «quién», pero no impone el turno: habría que implementar aparte el orden de llegada, con búsquedas y recorridos adicionales. Además, elegir un elemento arbitrario de una lista enlazada cuesta O(n), mientras que en una cola el frente se consulta en O(1).

En resumen: la cola es la única de las tres estructuras que garantiza por sí misma el orden FIFO que exige el pase de lista, con las operaciones básicas en tiempo constante.

Modelo de datos y reglas de negocio

El programa se dividió en cuatro clases para separar la lógica de la interfaz:

- **Alumno:** representa a un alumno: nombre, fila, lugar y si asistió.
- **RegistroDia:** guarda la fotografía de una jornada terminada (día, distribución del aula, alumnos con su asiento y los totales de asistencia y faltas). Trabaja con copias, por lo que empezar el día 2 no altera el día 1.
- **GestorAula:** contiene la cola, la matriz de asientos, la lista de ausentes y el historial; aplica las validaciones y la persistencia. No depende de Swing, lo que permite probarla sin ventanas.
- **GestionAsientosAula:** interfaz gráfica con cuatro pantallas (inicio, gestión, resumen y comparativa). Solo llama a los métodos del gestor.

Reglas implementadas:

- **Rango:** la fila debe existir y la silla debe existir dentro de esa fila;
- **Disponibilidad:** una silla ocupada no puede asignarse a otro alumno; el alumno permanece en la cola para que elija otra;
- **Corrección:** un alumno ausente puede recibir silla después, y entonces deja de ser falta;
- **Orden de jornadas:** el día 2 solo se habilita cuando el día 1 está cerrado, y un día no puede registrarse dos veces;
- **Capacidad:** deben existir al menos tantas sillas como alumnos en la lista.

Diagrama de casos de uso

El diagrama muestra los dos actores del caso: el docente, que opera el sistema, y el alumno, que elige su asiento. Los casos de uso corresponden uno a uno con las pantallas y acciones del programa.

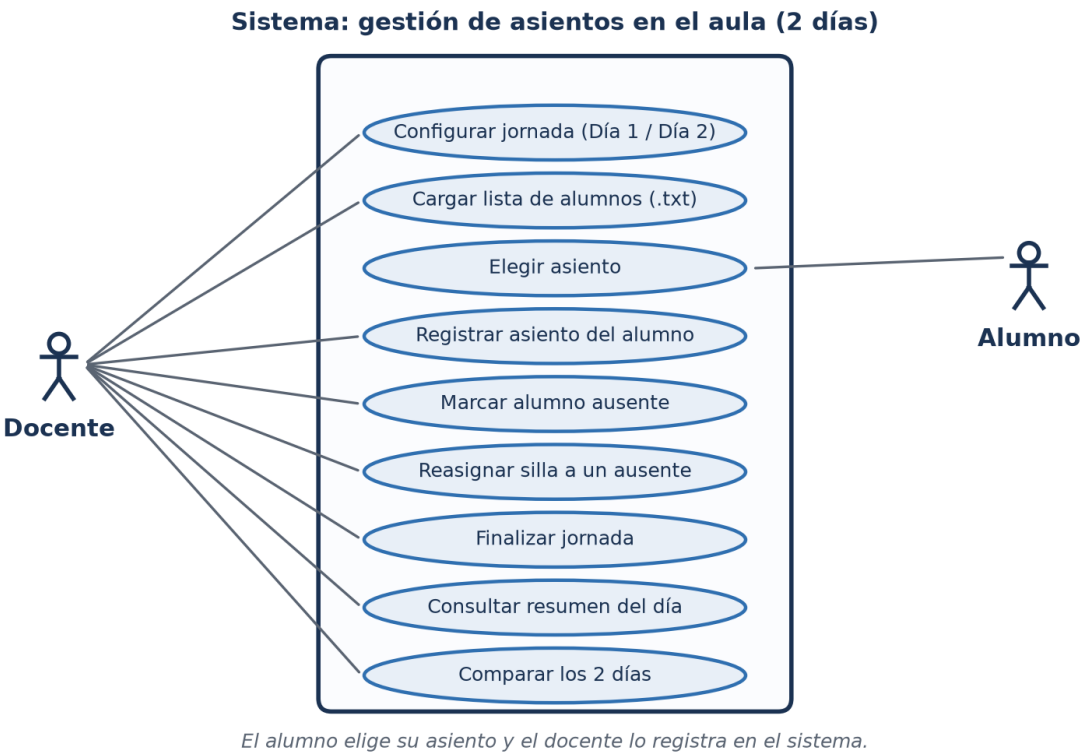


Figura 1. Diagrama de casos de uso del sistema.

Diagrama de clases

El diagrama refleja la separación entre interfaz y lógica. La interfaz usa un único objeto GestorAula; el gestor mantiene la cola de alumnos, la matriz de asientos y el historial; cada jornada cerrada se guarda como un RegistroDia que conserva copias de los alumnos.

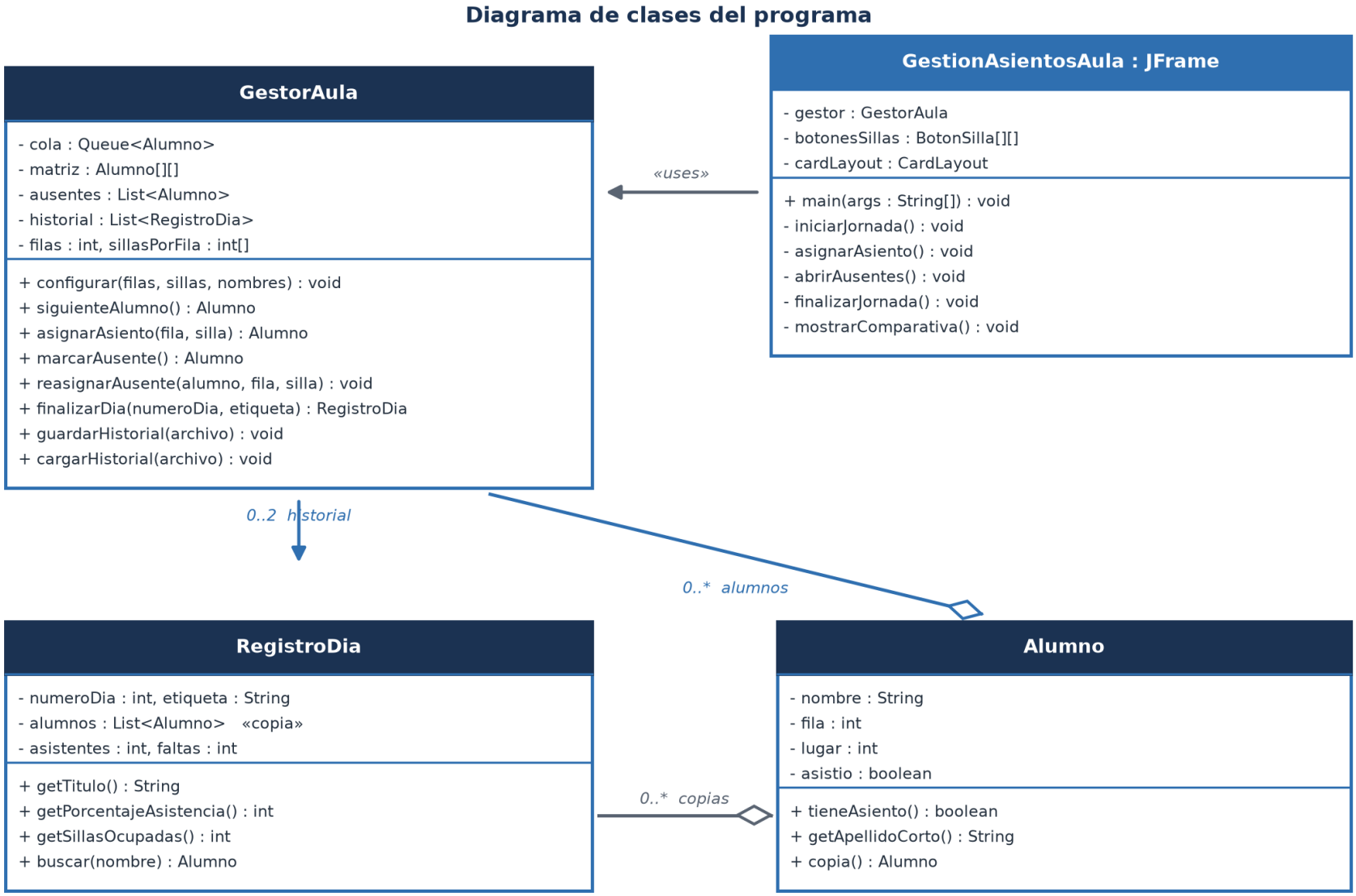


Figura 2. Diagrama de clases del programa.

2. Codificación

El programa se desarrolló en Java con la biblioteca Swing incluida en el JDK, sin dependencias externas. Se compila y ejecuta desde la carpeta del proyecto con:

```
javac -d build src/*.java
java -cp build GestionAsientosAula
```

A continuación se presenta el código fuente completo de cada clase, con numeración de líneas. El proyecto también está disponible en el repositorio entregado, junto con la carpeta de pruebas y las capturas de ejecución.

Alumno.java

Modelo del alumno. Conserva el nombre, el asiento elegido (fila y lugar) y la asistencia, y ofrece el método copia() que utiliza el historial.

```java
/**
 * Modelo de un alumno dentro del pase de lista.
 *
 * Guarda el lugar que eligio el alumno (fila y silla) y si asistio o no.
 * Cuando una jornada termina, se guarda una copia (metodo copia()) dentro
 * de RegistroDia para conservar el historial del dia sin que se modifique
 * despues.
 */
public class Alumno {

    private final String nombre;   // Apellidos, Nombre (tal como viene en el .txt)
    private int fila = -1;         // -1 significa "sin asiento asignado"
    private int lugar = -1;
    private boolean asistio = false;

    public Alumno(String nombre) {
        this.nombre = nombre.trim();
    }

    public String getNombre() {
        return nombre;
    }

    public int getFila() {
        return fila;
    }

    public void setFila(int fila) {
        this.fila = fila;
    }

    public int getLugar() {
        return lugar;
    }

    public void setLugar(int lugar) {
        this.lugar = lugar;
    }

    public boolean isAsistio() {
        return asistio;
    }

    public void setAsistio(boolean asistio) {
        this.asistio = asistio;
    }

    /** true si ya se le asigno una silla fisica en el aula. */
    public boolean tieneAsiento() {
        return fila > 0 && lugar > 0;
    }
```

```java
    /** Texto corto para dibujar dentro del boton de la silla. */
    public String getApellidoCorto() {
        int coma = nombre.indexOf(',');
        String apellido = (coma > 0) ? nombre.substring(0, coma) : nombre;
        return apellido.length() > 13 ? apellido.substring(0, 12) + "." : apellido;
    }

    /** Copia independiente, usada para congelar la informacion del dia. */
    public Alumno copia() {
        Alumno c = new Alumno(nombre);
        c.fila = fila;
        c.lugar = lugar;
        c.asistio = asistio;
        return c;
    }

    @Override
    public String toString() {
        return nombre;
    }
}
```

RegistroDia.java

Fotografía inmutable de una jornada finalizada. Calcula asistentes, faltas, porcentaje de asistencia y sillas ocupadas, y permite buscar a un alumno para la tabla comparativa.

```java
import java.util.ArrayList;
import java.util.List;

/**
 * Fotografia (snapshot) de una jornada ya finalizada.
 *
 * Se guarda por cada uno de los 2 dias del caso practico:
 * numero de dia, dia de la semana, distribucion del aula y el estado
 * final de cada alumno (fila, lugar y asistencia).
 *
 * Es inmutable hacia afuera: recibe copias de los alumnos, por lo que
 * empezar el dia siguiente no altera los registros anteriores.
 */
public class RegistroDia {

    private final int numeroDia;          // 1 o 2
    private final String etiqueta;        // "Lunes", "Martes", ...
    private final int filas;
    private final int[] sillasPorFila;
    private final List<Alumno> alumnos;   // copias con su asiento y asistencia
    private final int asistentes;
    private final int faltas;

    public RegistroDia(int numeroDia, String etiqueta, int filas,
                       int[] sillasPorFila, List<Alumno> alumnosDelDia) {
        this.numeroDia = numeroDia;
        this.etiqueta = etiqueta;
        this.filas = filas;
        this.sillasPorFila = sillasPorFila.clone();

        this.alumnos = new ArrayList<>();
        int asistieron = 0;
        for (Alumno a : alumnosDelDia) {
            Alumno copia = a.copia();
            alumnos.add(copia);
            if (copia.isAsistio()) {
                asistieron++;
            }
        }
        this.asistentes = asistieron;
        this.faltas = alumnosDelDia.size() - asistieron;
    }

    public int getNumeroDia() {
        return numeroDia;
    }

    public String getEtiqueta() {
        return etiqueta;
    }

    public int getFilas() {
```

```
        return filas;
    }

    public int getSillasPorFila(int fila) {
        return sillasPorFila[fila];
    }

    public int[] getSillasPorFila() {
        return sillasPorFila.clone();
    }

    public List<Alumno> getAlumnos() {
        return new ArrayList<>(alumnos);
    }

    public int getTotalAlumnos() {
        return alumnos.size();
    }

    public int getAsistentes() {
        return asistentes;
    }

    public int getFaltas() {
        return faltas;
    }

    /** Porcentaje de asistencia redondeado, para las tarjetas del resumen. */
    public int getPorcentajeAsistencia() {
        if (alumnos.isEmpty()) {
            return 0;
        }
        return (int) Math.round(asistentes * 100.0 / alumnos.size());
    }

    public int getTotalSillas() {
        int total = 0;
        for (int s : sillasPorFila) {
            total += s;
        }
        return total;
    }

    /** Sillas realmente ocupadas (alumnos que asistieron y eligieron lugar). */
    public int getSillasOcupadas() {
        int ocupadas = 0;
        for (Alumno a : alumnos) {
            if (a.tieneAsiento() && a.isAsistio()) {
                ocupadas++;
            }
        }
        return ocupadas;
```

```
    }

    /** Busca a un alumno por nombre para armar la tabla comparativa. */
    public Alumno buscar(String nombre) {
        for (Alumno a : alumnos) {
            if (a.getNombre().equalsIgnoreCase(nombre)) {
                return a;
            }
        }
        return null;
    }

    public String getTitulo() {
        return "Dia " + numeroDia + " - " + etiqueta;
    }
}
```

GestorAula.java

Núcleo del caso práctico: declara la cola FIFO, la matriz de asientos y el historial; implementa la asignación, la marcación de ausentes, la reasignación, el cierre de jornada, las validaciones y la persistencia en archivo.

```java
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * Logica del caso practico: "Gestion de asientos en el aula, por dia,
 * considerar 2 dias; el alumno decide que asiento utilizar".
 *
 * Estructura lineal usada: COLA (Queue sobre LinkedList), principio FIFO.
 *  - offer()  -> encola al alumno al final de la fila virtual.
 *  - peek()   -> muestra quien sigue sin sacarlo (tarjeta de turno).
 *  - poll()   -> saca al alumno cuando ya eligio silla o se marco ausente.
 *
 * La interfaz grafica (GestionAsientosAula) solo llama a estos metodos:
 * aqui no hay nada de Swing, lo que permite probar la logica por separado.
 */
public class GestorAula {

    /** El caso practico pide exactamente 2 dias. */
    public static final int TOTAL_DIAS = 2;

    private int filas;
    private int[] sillasPorFila = new int[0];
    private Alumno[][] matriz;                // matrizAsientos[fila][silla]

    private final Queue<Alumno> cola = new LinkedList<>();
    private final List<Alumno> ausentes = new ArrayList<>();
    private final List<Alumno> todos = new ArrayList<>();
    private final List<RegistroDia> historial = new ArrayList<>();

    // ------------------------------------------------------------------
    // Configuracion del salon
    // ------------------------------------------------------------------

    /**
     * Prepara una jornada nueva: guarda la distribucion del aula, reinicia la
     * matriz de asientos y vuelve a encolar a todos los alumnos de la lista.
     */
    public void configurar(int filas, int[] sillasPorFila, List<String> nombres) {
        this.filas = filas;
        this.sillasPorFila = sillasPorFila.clone();

        this.matriz = new Alumno[filas][maxSillas()];
        this.cola.clear();
```

```
53          this.ausentes.clear();
54          this.todos.clear();
55
56          for (String nombre : nombres) {
57              Alumno alumno = new Alumno(nombre);
58              todos.add(alumno);
59              cola.offer(alumno);          // FIFO: se encola al final
60          }
61      }
62
63      public int getFilas() {
64          return filas;
65      }
66
67      public int getSillasEnFila(int fila) {
68          return sillasPorFila[fila];
69      }
70
71      public int[] getSillasPorFila() {
72          return sillasPorFila.clone();
73      }
74
75      public int maxSillas() {
76          int max = 0;
77          for (int s : sillasPorFila) {
78              max = Math.max(max, s);
79          }
80          return max;
81      }
82
83      public Alumno[][] getMatriz() {
84          return matriz;
85      }
86
87      public List<Alumno> getAusentes() {
88          return new ArrayList<>(ausentes);
89      }
90
91      public List<Alumno> getTodos() {
92          return new ArrayList<>(todos);
93      }
94
95      public List<RegistroDia> getHistorial() {
96          return new ArrayList<>(historial);
97      }
98
99      // ------------------------------------------------------------------
100     // Operaciones de la cola
101     // ------------------------------------------------------------------
102
103     /** Alumno al frente de la cola (peek): el que aparece en la tarjeta. */
104     public Alumno siguienteAlumno() {
```

```
105         return cola.peek();
106     }
107
108     public int getPendientes() {
109         return cola.size();
110     }
111
112     public int getProcesados() {
113         return todos.size() - cola.size();
114     }
115
116     public int getTotalAlumnos() {
117         return todos.size();
118     }
119
120     public int getOcupadas() {
121         int ocupadas = 0;
122         for (int f = 0; f < filas; f++) {
123             for (int s = 0; s < sillasPorFila[f]; s++) {
124                 if (matriz[f][s] != null) {
125                     ocupadas++;
126                 }
127             }
128         }
129         return ocupadas;
130     }
131
132     public int getTotalSillas() {
133         int total = 0;
134         for (int s : sillasPorFila) {
135             total += s;
136         }
137         return total;
138     }
139
140     public int getLibres() {
141         return getTotalSillas() - getOcupadas();
142     }
143
144     /** Valida fila/silla y devuelve el alumno que ocupa ese lugar (o null). */
145     public Alumno ocupante(int fila, int lugar) {
146         return matriz[fila - 1][lugar - 1];
147     }
148
149     /**
150      * Asigna al alumno que esta al frente de la cola el asiento elegido.
151      * Se valida rango y disponibilidad antes de sacarlo de la cola, para no
152      * perder al alumno si el dato es incorrecto.
153      */
154     public Alumno asignarAsiento(int fila, int lugar) throws ReglaNegocioException {
155         Alumno actual = cola.peek();
156         if (actual == null) {
```

```
            throw new ReglaNegocioException("La cola esta vacia: todos los alumnos ya fueron procesados.");
        }
        validarAsientoLibre(fila, lugar);

        cola.poll();                    // FIFO: sale el primero de la fila
        actual.setFila(fila);
        actual.setLugar(lugar);
        actual.setAsistio(true);
        matriz[fila - 1][lugar - 1] = actual;
        return actual;
    }

    /** Saca de la cola al alumno actual y lo agrega a la lista de ausentes. */
    public Alumno marcarAusente() throws ReglaNegocioException {
        Alumno actual = cola.peek();
        if (actual == null) {
            throw new ReglaNegocioException("La cola esta vacia: no hay alumnos pendientes.");
        }
        cola.poll();
        actual.setAsistio(false);
        actual.setFila(-1);
        actual.setLugar(-1);
        ausentes.add(actual);
        return actual;
    }

    /**
     * Permite corregir: un alumno marcado como ausente llega tarde y elige
     * una silla. Sale de la lista de ausentes y se marca como asistente.
     */
    public void reasignarAusente(Alumno alumno, int fila, int lugar) throws ReglaNegocioException {
        if (alumno == null) {
            throw new ReglaNegocioException("Seleccione un alumno de la lista de ausentes.");
        }
        validarAsientoLibre(fila, lugar);

        alumno.setFila(fila);
        alumno.setLugar(lugar);
        alumno.setAsistio(true);
        matriz[fila - 1][lugar - 1] = alumno;
        ausentes.remove(alumno);
    }

    /** Reglas compartidas por la asignacion normal y la reasignacion. */
    private void validarAsientoLibre(int fila, int lugar) throws ReglaNegocioException {
        if (fila < 1 || fila > filas) {
            throw new ReglaNegocioException("La fila debe estar entre 1 y " + filas + ".");
        }
        if (lugar < 1 || lugar > sillasPorFila[fila - 1]) {
            throw new ReglaNegocioException("En la fila " + fila + " la silla debe estar entre 1 y "
                    + sillasPorFila[fila - 1] + ".");
        }
```

```
209         Alumno ocupante = matriz[fila - 1][lugar - 1];
210         if (ocupante != null) {
211             throw new ReglaNegocioException("La silla F" + fila + "-S" + lugar
212                     + " ya esta ocupada por " + ocupante.getNombre() + ".");
213         }
214     }
215 
216     // ------------------------------------------------------------------
217     // Cierre de la jornada
218     // ------------------------------------------------------------------
219 
220     /**
221      * Cierra el dia: los alumnos que quedaron en la cola cuentan como falta,
222      * y se guarda la fotografia del dia en el historial.
223      */
224     public RegistroDia finalizarDia(int numeroDia, String etiqueta) {
225         while (!cola.isEmpty()) {
226             Alumno pendiente = cola.poll();
227             pendiente.setAsistio(false);
228             if (!ausentes.contains(pendiente)) {
229                 ausentes.add(pendiente);
230             }
231         }
232         RegistroDia registro = new RegistroDia(numeroDia, etiqueta, filas, sillasPorFila, todos);
233         historial.add(registro);
234         return registro;
235     }
236 
237     public boolean diaRegistrado(int numeroDia) {
238         for (RegistroDia r : historial) {
239             if (r.getNumeroDia() == numeroDia) {
240                 return true;
241             }
242         }
243         return false;
244     }
245 
246     public RegistroDia buscarDia(int numeroDia) {
247         for (RegistroDia r : historial) {
248             if (r.getNumeroDia() == numeroDia) {
249                 return r;
250             }
251         }
252         return null;
253     }
254 
255     // ------------------------------------------------------------------
256     // Persistencia
257     // ------------------------------------------------------------------
258 
259     /** Lee un alumno por linea; ignora lineas vacias. */
260     public static List<String> leerAlumnos(File archivo) throws IOException {
```

```
        List<String> nombres = new ArrayList<>();
        try (BufferedReader br = new BufferedReader(new FileReader(archivo))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                linea = linea.trim();
                if (!linea.isEmpty() && !linea.startsWith("#")) {
                    nombres.add(linea);
                }
            }
        }
        return nombres;
    }

    /** Lista de ejemplo usada cuando no se selecciona ningun archivo. */
    public static List<String> alumnosPorDefecto() {
        List<String> nombres = new ArrayList<>();
        nombres.add("Alarcon Valencia, Carlos Adrian");
        nombres.add("Barrientos Marina, Carlos David");
        nombres.add("Cordoba Argon, Maria");
        nombres.add("Dominguez Perez, Juan");
        nombres.add("Gomez Ruiz, Laura");
        nombres.add("Hernandez Lopez, Pedro");
        return nombres;
    }

    /**
     * Guarda el historial en un archivo TSV (se abre con Excel).
     * Una fila por alumno y por dia, con la distribucion del aula incluida.
     */
    public void guardarHistorial(File destino) throws IOException {
        try (PrintWriter out = new PrintWriter(new BufferedWriter(new FileWriter(destino)))) {
            out.println("# numeroDia\tetiqueta\tfilas\tsillasPorFila\tnombre\tfila\tlugar\tasistio");
            for (RegistroDia r : historial) {
                String sillas;
                int[] arreglo = r.getSillasPorFila();
                StringBuilder sb = new StringBuilder();
                for (int i = 0; i < arreglo.length; i++) {
                    if (i > 0) {
                        sb.append(',');
                    }
                    sb.append(arreglo[i]);
                }
                sillas = sb.toString();

                for (Alumno a : r.getAlumnos()) {
                    out.println(r.getNumeroDia() + "\t" + r.getEtiqueta() + "\t" + r.getFilas()
                            + "\t" + sillas + "\t" + a.getNombre() + "\t" + a.getFila()
                            + "\t" + a.getLugar() + "\t" + (a.isAsistio() ? "1" : "0"));
                }
            }
        }
    }
```

```java
    /** Reconstruye el historial guardado con guardarHistorial(). */
    public void cargarHistorial(File origen) throws IOException {
        if (origen == null || !origen.exists()) {
            return;
        }
        List<RegistroDia> cargados = new ArrayList<>();
        int diaActual = -1;
        String etiquetaActual = "";
        int filasActual = 0;
        int[] sillasActual = new int[0];
        List<Alumno> alumnosActual = new ArrayList<>();

        try (BufferedReader br = new BufferedReader(new FileReader(origen))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                if (linea.trim().isEmpty() || linea.startsWith("#")) {
                    continue;
                }
                String[] campos = linea.split("\t");
                if (campos.length < 8) {
                    continue;
                }
                int numero = Integer.parseInt(campos[0]);
                if (numero != diaActual) {
                    if (diaActual != -1) {
                        cargados.add(new RegistroDia(diaActual, etiquetaActual, filasActual, sillasActual, alumnosActual));
                    }
                    diaActual = numero;
                    etiquetaActual = campos[1];
                    filasActual = Integer.parseInt(campos[2]);
                    String[] partes = campos[3].split(",");
                    sillasActual = new int[partes.length];
                    for (int i = 0; i < partes.length; i++) {
                        sillasActual[i] = Integer.parseInt(partes[i].trim());
                    }
                    alumnosActual = new ArrayList<>();
                }
                Alumno a = new Alumno(campos[4]);
                a.setFila(Integer.parseInt(campos[5]));
                a.setLugar(Integer.parseInt(campos[6]));
                a.setAsistio("1".equals(campos[7]));
                alumnosActual.add(a);
            }
        }
        if (diaActual != -1) {
            cargados.add(new RegistroDia(diaActual, etiquetaActual, filasActual, sillasActual, alumnosActual));
        }
        historial.clear();
        historial.addAll(cargados);
    }
```

```java
    // ------------------------------------------------------------------
    // Excepcion de reglas de negocio (mensajes para el usuario)
    // ------------------------------------------------------------------

    public static class ReglaNegocioException extends Exception {
        public ReglaNegocioException(String mensaje) {
            super(mensaje);
        }
    }
}
```

GestionAsientosAula.java

Interfaz gráfica. Contiene la configuración de la jornada, la tarjeta del alumno en turno, el mapa de asientos, el resumen del día y la comparativa de las dos jornadas.

```java
import java.awt.BasicStroke;
import java.awt.BorderLayout;
import java.awt.CardLayout;
import java.awt.Color;
import java.awt.Component;
import java.awt.Cursor;
import java.awt.Dimension;
import java.awt.Font;
import java.awt.FontMetrics;
import java.awt.GradientPaint;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.GridLayout;
import java.awt.Insets;
import java.awt.RenderingHints;
import java.io.File;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

import javax.swing.BorderFactory;
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JDialog;
import javax.swing.JFileChooser;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JList;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JSpinner;
import javax.swing.JTable;
import javax.swing.JTextField;
import javax.swing.ListSelectionModel;
import javax.swing.SpinnerNumberModel;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.UIManager;
import javax.swing.border.EmptyBorder;
import javax.swing.filechooser.FileNameExtensionFilter;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.DefaultTableModel;

/**
 * Caso practico Unidad 2 - Estructuras de Datos.
 * "Gestion de asientos en el aula, por dia, considerar 2 dias.
 *  El alumno decide que asiento utilizar."
```

```
 *
 * Estructura lineal: COLA (FIFO) implementada con Queue/LinkedList.
 *
 * Pantallas:
 *   1. Inicio      -> jornada (dia 1 o dia 2), dia de la semana, filas, sillas,
 *                     lista de alumnos (.txt).
 *   2. Gestion     -> tarjeta del alumno al frente de la cola y mapa de sillas.
 *   3. Resumen     -> totales del dia y mapa de asientos en solo lectura.
 *   4. Comparativa -> tabla y graficas de los 2 dias.
 */
public class GestionAsientosAula extends JFrame {

    // ------------------------- Paleta y tipografias -------------------------
    private static final Color AZUL = new Color(0x1B3252);
    private static final Color AZUL_MEDIO = new Color(0x2F6FB0);
    private static final Color FONDO = new Color(0xF1F4F8);
    private static final Color VERDE = new Color(0x2E9E5B);
    private static final Color ROJO = new Color(0xD64545);
    private static final Color GRIS_TEXTO = new Color(0x5A6472);
    private static final Color TARJETA = Color.WHITE;

    private static final Font FUENTE_TITULO = new Font("Segoe UI", Font.BOLD, 26);
    private static final Font FUENTE_SUBTITULO = new Font("Segoe UI", Font.PLAIN, 14);
    private static final Font FUENTE_SECCION = new Font("Segoe UI", Font.BOLD, 16);
    private static final Font FUENTE_NORMAL = new Font("Segoe UI", Font.PLAIN, 13);
    private static final Font FUENTE_BOTON = new Font("Segoe UI", Font.BOLD, 13);

    // ---------------------------- Estado ----------------------------------
    private final GestorAula gestor = new GestorAula();
    private final File archivoHistorial = new File("historial_asientos.tsv");

    private CardLayout cardLayout;
    private JPanel contenedor;

    // Pantalla de inicio
    private JComboBox<String> cbJornada;
    private JComboBox<String> cbDiaSemana;
    private JSpinner spFilas;
    private JPanel panelSillasPorFila;
    private final List<JSpinner> spinnersSillas = new ArrayList<>();
    private JLabel lblArchivo;
    private JLabel lblPieInicio;
    private File archivoAlumnos;

    // Pantalla de gestion
    private JLabel lblTurnoNumero;
    private JLabel lblNombreAlumno;
    private JLabel lblEstadoDia;
    private JTextField txtFila;
    private JTextField txtLugar;
    private Boton btnAceptar;
    private Boton btnAusente;
```

```
105     private Boton btnVerAusentes;
106     private JPanel panelTarjeta;
107     private JPanel panelAula;
108     private JLabel lblEstadoBarra;
109     private BotonSilla[][] botonesSillas;
110     private JLabel lblContadorOcupadas;
111     private JLabel lblContadorLibres;
112     private JLabel lblContadorCola;
113     private Boton btnFinalizar;
114
115     // Pantalla de resumen
116     private JPanel panelResumen;
117
118     // Pantalla de comparativa
119     private JPanel panelComparativa;
120
121     private int diaEnCurso = 1;
122     private String diaSemanaEnCurso = "Lunes";
123
124     public GestionAsientosAula() {
125         setTitle("Gestion de Asientos en el Aula - Cola (FIFO) - Unidad 2");
126         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
127         setSize(1200, 780);
128         setMinimumSize(new Dimension(1060, 700));
129         setLocationRelativeTo(null);
130
131         cargarHistorialGuardado();
132
133         cardLayout = new CardLayout();
134         contenedor = new JPanel(cardLayout);
135         contenedor.setBackground(FONDO);
136
137         contenedor.add(crearPantallaInicio(), "inicio");
138         contenedor.add(crearPantallaGestion(), "gestion");
139         contenedor.add(crearPantallaResumen(), "resumen");
140         contenedor.add(crearPantallaComparativa(), "comparativa");
141
142         add(contenedor);
143         mostrarInicio();
144     }
145
146     // ========================================================================
147     // 1. PANTALLA DE INICIO
148     // ========================================================================
149
150     private JPanel crearPantallaInicio() {
151         JPanel pantalla = new JPanel(new BorderLayout());
152         pantalla.setBackground(FONDO);
153         pantalla.add(crearEncabezado(), BorderLayout.NORTH);
154
155         JPanel centro = new JPanel(new GridBagLayout());
156         centro.setBackground(FONDO);
```

```
        centro.setBorder(new EmptyBorder(18, 18, 18, 18));

        JPanel tarjeta = tarjetaBlanca();
        tarjeta.setLayout(new BoxLayout(tarjeta, BoxLayout.Y_AXIS));
        tarjeta.setBorder(new EmptyBorder(22, 28, 22, 28));
        tarjeta.setPreferredSize(new Dimension(820, 490));

        JLabel titulo = new JLabel("Configuracion de la jornada");
        titulo.setFont(FUENTE_SECCION);
        titulo.setForeground(AZUL);
        titulo.setAlignmentX(Component.LEFT_ALIGNMENT);
        tarjeta.add(titulo);

        JLabel ayuda = new JLabel("El caso practico considera 2 dias: registre el Dia 1 y despues el Dia 2.");
        ayuda.setFont(FUENTE_NORMAL);
        ayuda.setForeground(GRIS_TEXTO);
        ayuda.setAlignmentX(Component.LEFT_ALIGNMENT);
        ayuda.setBorder(new EmptyBorder(4, 0, 16, 0));
        tarjeta.add(ayuda);

        JPanel formulario = new JPanel(new GridBagLayout());
        formulario.setBackground(TARJETA);
        formulario.setAlignmentX(Component.LEFT_ALIGNMENT);
        formulario.setMaximumSize(new Dimension(Integer.MAX_VALUE, 400));        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(8, 4, 8, 14);
        gbc.anchor = GridBagConstraints.WEST;
        gbc.fill = GridBagConstraints.NONE;

        // Jornada
        gbc.gridx = 0;
        gbc.gridy = 0;
        formulario.add(etiquetaAncha("Jornada a registrar:"), gbc);
        cbJornada = new JComboBox<>(new String[]{"Dia 1", "Dia 2"});
        cbJornada.setFont(FUENTE_NORMAL);
        gbc.gridx = 1;
        formulario.add(cbJornada, gbc);

        // Dia de la semana
        gbc.gridx = 0;
        gbc.gridy = 1;
        formulario.add(etiquetaAncha("Dia de la semana:"), gbc);
        cbDiaSemana = new JComboBox<>(new String[]{"Lunes", "Martes", "Miercoles",
                "Jueves", "Viernes", "Sabado"});
        cbDiaSemana.setFont(FUENTE_NORMAL);
        gbc.gridx = 1;
        formulario.add(cbDiaSemana, gbc);

        // Filas
        gbc.gridx = 0;
        gbc.gridy = 2;
        formulario.add(etiquetaAncha("Cantidad de filas:"), gbc);
        spFilas = new JSpinner(new SpinnerNumberModel(3, 1, 10, 1));
```

```
209         spFilas.setFont(FUENTE_NORMAL);
210         spFilas.setPreferredSize(new Dimension(80, 32));
211         spFilas.addChangeListener(e -> construirSpinnersSillas());
212         gbc.gridx = 1;
213         formulario.add(spFilas, gbc);
214 
215         // Sillas por fila
216         gbc.gridx = 0;
217         gbc.gridy = 3;
218         gbc.anchor = GridBagConstraints.NORTHWEST;
219         formulario.add(etiquetaAncha("Sillas por fila:"), gbc);
220         panelSillasPorFila = new JPanel(new GridLayout(0, 3, 8, 6));
221         panelSillasPorFila.setBackground(TARJETA);
222         gbc.gridx = 1;
223         gbc.fill = GridBagConstraints.HORIZONTAL;
224         gbc.weightx = 1;
225         formulario.add(panelSillasPorFila, gbc);
226 
227         // Archivo
228         gbc.gridx = 0;
229         gbc.gridy = 4;
230         gbc.fill = GridBagConstraints.HORIZONTAL;
231         formulario.add(etiquetaAncha("Lista de alumnos (.txt):"), gbc);
232         JPanel filaArchivo = new JPanel(new BorderLayout(8, 0));
233         filaArchivo.setBackground(TARJETA);
234         Boton btnArchivo = new Boton("Seleccionar archivo...", AZUL_MEDIO);
235         btnArchivo.addActionListener(e -> seleccionarArchivo());
236         lblArchivo = new JLabel("Sin archivo: se usara la lista de ejemplo");
237         lblArchivo.setFont(new Font("Segoe UI", Font.ITALIC, 12));
238         lblArchivo.setForeground(GRIS_TEXTO);
239         filaArchivo.add(btnArchivo, BorderLayout.WEST);
240         filaArchivo.add(lblArchivo, BorderLayout.CENTER);
241         gbc.gridx = 1;
242         gbc.weightx = 1;
243         formulario.add(filaArchivo, gbc);
244 
245         tarjeta.add(formulario);
246         tarjeta.add(Box.createVerticalStrut(14));
247 
248         Boton btnComenzar = new Boton("Comenzar pase de lista", VERDE);
249         btnComenzar.setFont(new Font("Segoe UI", Font.BOLD, 15));
250         btnComenzar.setAlignmentX(Component.LEFT_ALIGNMENT);
251         btnComenzar.setMaximumSize(new Dimension(Integer.MAX_VALUE, 46));
252         btnComenzar.addActionListener(e -> iniciarJornada());
253         tarjeta.add(btnComenzar);
254 
255         lblPieInicio = new JLabel(" ");
256         lblPieInicio.setFont(FUENTE_NORMAL);
257         lblPieInicio.setForeground(GRIS_TEXTO);
258         lblPieInicio.setAlignmentX(Component.LEFT_ALIGNMENT);
259         lblPieInicio.setBorder(new EmptyBorder(12, 0, 0, 0));
260         tarjeta.add(lblPieInicio);
```

```
        centro.add(tarjeta);
        pantalla.add(centro, BorderLayout.CENTER);

        construirSpinnersSillas();
        return pantalla;
    }

    /** Crea un spinner de sillas por cada fila del salon. */
    private void construirSpinnersSillas() {
        int filas = (Integer) spFilas.getValue();
        panelSillasPorFila.removeAll();
        spinnersSillas.clear();

        for (int i = 0; i < filas; i++) {
            JPanel caja = new JPanel(new BorderLayout(6, 0));
            caja.setBackground(TARJETA);
            JLabel etiquetaFila = new JLabel("Fila " + (i + 1) + ":");
            etiquetaFila.setFont(FUENTE_NORMAL);
            etiquetaFila.setForeground(GRIS_TEXTO);
            JSpinner spinner = new JSpinner(new SpinnerNumberModel(4, 1, 12, 1));
            spinner.setFont(FUENTE_NORMAL);
            spinner.setPreferredSize(new Dimension(60, 30));
            caja.add(etiquetaFila, BorderLayout.WEST);
            caja.add(spinner, BorderLayout.CENTER);
            panelSillasPorFila.add(caja);
            spinnersSillas.add(spinner);
        }
        panelSillasPorFila.revalidate();
        panelSillasPorFila.repaint();
        actualizarPieInicio();
    }

    private void actualizarPieInicio() {
        int sillas = 0;
        for (JSpinner s : spinnersSillas) {
            sillas += (Integer) s.getValue();
        }
        int dias = gestor.getHistorial().size();
        lblPieInicio.setText("Aula configurada con " + spinnersSillas.size() + " filas y "
                + sillas + " sillas  |  Dias registrados: " + dias + " de " + GestorAula.TOTAL_DIAS);
    }

    private void seleccionarArchivo() {
        JFileChooser chooser = new JFileChooser();
        chooser.setDialogTitle("Selecciona la lista de alumnos (.txt)");
        chooser.setFileFilter(new FileNameExtensionFilter("Archivos de texto (*.txt)", "txt"));
        if (chooser.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {
            archivoAlumnos = chooser.getSelectedFile();
            lblArchivo.setText(" " + archivoAlumnos.getName());
        }
    }
```

```java
    /** Lee el formulario y arranca la jornada elegida. */
    private void iniciarJornada() {
        int numeroDia = cbJornada.getSelectedIndex() + 1;
        String diaSemana = (String) cbDiaSemana.getSelectedItem();

        if (gestor.diaRegistrado(numeroDia)) {
            aviso("El Dia " + numeroDia + " ya fue registrado. Elija el otro dia para continuar.",
                    "Jornada ya registrada", JOptionPane.WARNING_MESSAGE);
            return;
        }
        if (numeroDia == 2 && !gestor.diaRegistrado(1)) {
            aviso("Primero debe finalizar el Dia 1 para poder registrar el Dia 2.",
                    "Orden de las jornadas", JOptionPane.WARNING_MESSAGE);
            return;
        }

        int filas = (Integer) spFilas.getValue();
        int[] sillas = new int[filas];
        for (int i = 0; i < filas; i++) {
            sillas[i] = (Integer) spinnersSillas.get(i).getValue();
        }

        List<String> nombres;
        try {
            nombres = (archivoAlumnos != null && archivoAlumnos.exists())
                    ? GestorAula.leerAlumnos(archivoAlumnos)
                    : GestorAula.alumnosPorDefecto();
        } catch (IOException ex) {
            aviso("No se pudo leer el archivo: " + ex.getMessage(), "Error de lectura",
                    JOptionPane.ERROR_MESSAGE);
            nombres = GestorAula.alumnosPorDefecto();
        }

        if (nombres.isEmpty()) {
            aviso("La lista de alumnos esta vacia.", "Lista vacia", JOptionPane.WARNING_MESSAGE);
            return;
        }
        if (nombres.size() > sumar(sillas)) {
            aviso("Hay " + nombres.size() + " alumnos y solo " + sumar(sillas)
                    + " sillas. Aumente las sillas o revise la lista.", "Faltan sillas",
                    JOptionPane.WARNING_MESSAGE);
            return;
        }

        diaEnCurso = numeroDia;
        diaSemanaEnCurso = diaSemana;
        gestor.configurar(filas, sillas, nombres);

        lblEstadoDia.setText("Dia " + numeroDia + " de " + GestorAula.TOTAL_DIAS
                + "  -  " + diaSemana);
        dibujarAula();
```

```
365         actualizarTarjeta();
366         cardLayout.show(contenedor, "gestion");
367     }
368 
369     private int sumar(int[] valores) {
370         int total = 0;
371         for (int v : valores) {
372             total += v;
373         }
374         return total;
375     }
376 
377     // ========================================================================
378     // 2. PANTALLA DE GESTION
379     // ========================================================================
380 
381     private JPanel crearPantallaGestion() {
382         JPanel pantalla = new JPanel(new BorderLayout());
383         pantalla.setBackground(FONDO);
384         pantalla.add(crearEncabezado(), BorderLayout.NORTH);
385 
386         // ---- Tarjeta del alumno en turno (izquierda) ----
387         panelTarjeta = tarjetaBlanca();
388         panelTarjeta.setLayout(new BoxLayout(panelTarjeta, BoxLayout.Y_AXIS));
389         panelTarjeta.setBorder(new EmptyBorder(18, 18, 18, 18));
390         panelTarjeta.setPreferredSize(new Dimension(330, 0));
391 
392         JLabel tituloTarjeta = new JLabel("Turno en cola (FIFO)");
393         tituloTarjeta.setFont(FUENTE_SECCION);
394         tituloTarjeta.setForeground(AZUL);
395         tituloTarjeta.setAlignmentX(Component.LEFT_ALIGNMENT);
396         panelTarjeta.add(tituloTarjeta);
397 
398         lblTurnoNumero = new JLabel("Alumno 1 de 1");
399         lblTurnoNumero.setFont(FUENTE_NORMAL);
400         lblTurnoNumero.setForeground(GRIS_TEXTO);
401         lblTurnoNumero.setAlignmentX(Component.LEFT_ALIGNMENT);
402         lblTurnoNumero.setBorder(new EmptyBorder(2, 0, 10, 0));
403         panelTarjeta.add(lblTurnoNumero);
404 
405         lblNombreAlumno = new JLabel("<html><b>Alumno:</b><br>-</html>");
406         lblNombreAlumno.setFont(new Font("Segoe UI", Font.PLAIN, 19));
407         lblNombreAlumno.setForeground(AZUL);
408         lblNombreAlumno.setAlignmentX(Component.LEFT_ALIGNMENT);
409         lblNombreAlumno.setBorder(new EmptyBorder(0, 0, 16, 0));
410         panelTarjeta.add(lblNombreAlumno);
411 
412         JPanel formulario = new JPanel(new GridBagLayout());
413         formulario.setBackground(TARJETA);
414         formulario.setAlignmentX(Component.LEFT_ALIGNMENT);
415         GridBagConstraints gbc = new GridBagConstraints();
416         gbc.insets = new Insets(5, 0, 5, 8);
```

```
417         gbc.fill = GridBagConstraints.HORIZONTAL;
418
419         gbc.gridx = 0;
420         gbc.gridy = 0;
421         formulario.add(etiqueta("Fila:"), gbc);
422         txtFila = new JTextField();
423         txtFila.setFont(FUENTE_NORMAL);
424         txtFila.setPreferredSize(new Dimension(90, 32));
425         gbc.gridx = 1;
426         formulario.add(txtFila, gbc);
427
428         gbc.gridx = 0;
429         gbc.gridy = 1;
430         formulario.add(etiqueta("Silla:"), gbc);
431         txtLugar = new JTextField();
432         txtLugar.setFont(FUENTE_NORMAL);
433         txtLugar.setPreferredSize(new Dimension(90, 32));
434         gbc.gridx = 1;
435         formulario.add(txtLugar, gbc);
436         panelTarjeta.add(formulario);
437         formulario.setMaximumSize(new Dimension(Integer.MAX_VALUE,
438                 formulario.getPreferredSize().height));
439
440         JLabel pista = new JLabel("<html>Tambien puede hacer clic en una<br>silla libre del mapa.</html>");
441         pista.setFont(new Font("Segoe UI", Font.ITALIC, 11));
442         pista.setForeground(GRIS_TEXTO);
443         pista.setAlignmentX(Component.LEFT_ALIGNMENT);
444         pista.setBorder(new EmptyBorder(6, 0, 12, 0));
445         panelTarjeta.add(pista);
446
447         btnAceptar = new Boton("Aceptar asiento", VERDE);
448         btnAceptar.setAlignmentX(Component.LEFT_ALIGNMENT);
449         btnAceptar.setMaximumSize(new Dimension(Integer.MAX_VALUE, 40));
450         btnAceptar.addActionListener(e -> asignarAsiento());
451         panelTarjeta.add(btnAceptar);
452         panelTarjeta.add(Box.createVerticalStrut(8));
453
454         btnAusente = new Boton("Marcar ausente", ROJO);
455         btnAusente.setAlignmentX(Component.LEFT_ALIGNMENT);
456         btnAusente.setMaximumSize(new Dimension(Integer.MAX_VALUE, 40));
457         btnAusente.addActionListener(e -> marcarAusente());
458         panelTarjeta.add(btnAusente);
459         panelTarjeta.add(Box.createVerticalStrut(8));
460
461         btnVerAusentes = new Boton("Ver lista de ausentes", AZUL_MEDIO);
462         btnVerAusentes.setAlignmentX(Component.LEFT_ALIGNMENT);
463         btnVerAusentes.setMaximumSize(new Dimension(Integer.MAX_VALUE, 40));
464         btnVerAusentes.addActionListener(e -> abrirAusentes());
465         panelTarjeta.add(btnVerAusentes);
466
467         panelTarjeta.add(Box.createVerticalGlue());
```

```
469         // ---- Mapa del aula (centro) ----
470         panelAula = new JPanel();
471         panelAula.setBackground(FONDO);
472         JScrollPane scroll = new JScrollPane(panelAula);
473         scroll.setBorder(tarjetaBlanca().getBorder());
474         scroll.getVerticalScrollBar().setUnitIncrement(16);
475         scroll.setBackground(FONDO);
476 
477         JPanel centro = new JPanel(new BorderLayout(14, 0));
478         centro.setBackground(FONDO);
479         centro.setBorder(new EmptyBorder(12, 16, 12, 16));
480         centro.add(panelTarjeta, BorderLayout.WEST);
481         centro.add(scroll, BorderLayout.CENTER);
482 
483         // ---- Barra inferior ----
484         JPanel barra = new JPanel(new BorderLayout());
485         barra.setBackground(TARJETA);
486         barra.setBorder(new EmptyBorder(10, 16, 10, 16));
487 
488         JPanel contadores = new JPanel(new GridLayout(1, 3, 18, 0));
489         contadores.setBackground(TARJETA);
490         lblContadorOcupadas = contador("Ocupadas: 0");
491         lblContadorLibres = contador("Libres: 0");
492         lblContadorCola = contador("En cola: 0");
493         contadores.add(lblContadorOcupadas);
494         contadores.add(lblContadorLibres);
495         contadores.add(lblContadorCola);
496 
497         lblEstadoBarra = new JLabel(" ");
498         lblEstadoBarra.setFont(FUENTE_NORMAL);
499         lblEstadoBarra.setForeground(GRIS_TEXTO);
500         lblEstadoBarra.setBorder(new EmptyBorder(0, 24, 0, 12));
501 
502         btnFinalizar = new Boton("Finalizar jornada", ROJO);
503         btnFinalizar.setPreferredSize(new Dimension(180, 40));
504         btnFinalizar.addActionListener(e -> finalizarJornada());
505 
506         barra.add(contadores, BorderLayout.WEST);
507         barra.add(lblEstadoBarra, BorderLayout.CENTER);
508         barra.add(btnFinalizar, BorderLayout.EAST);
509 
510         JPanel sur = new JPanel(new BorderLayout());
511         sur.setBackground(TARJETA);
512         JLabel dia = new JLabel("  ");
513         lblEstadoDia = dia;
514         dia.setFont(FUENTE_SECCION);
515         dia.setForeground(AZUL);
516         JPanel franja = new JPanel(new BorderLayout());
517         franja.setBackground(new Color(0xE6EEF8));
518         franja.setBorder(new EmptyBorder(8, 16, 8, 16));
519         franja.add(dia, BorderLayout.WEST);
520         sur.add(franja, BorderLayout.NORTH);
```

```
        sur.add(barra, BorderLayout.SOUTH);

        pantalla.add(centro, BorderLayout.CENTER);
        pantalla.add(sur, BorderLayout.SOUTH);
        return pantalla;
    }

    private void dibujarAula() {
        panelAula.removeAll();
        panelAula.setLayout(new BoxLayout(panelAula, BoxLayout.Y_AXIS));
        panelAula.setBorder(new EmptyBorder(14, 14, 14, 14));
        botonesSillas = new BotonSilla[gestor.getFilas()][gestor.maxSillas()];

        for (int f = 0; f < gestor.getFilas(); f++) {
            JPanel fila = new JPanel(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT, 8, 6));
            fila.setBackground(FONDO);
            fila.setAlignmentX(Component.LEFT_ALIGNMENT);
            fila.setMaximumSize(new Dimension(Integer.MAX_VALUE, 74));

            JLabel etiquetaFila = new JLabel("Fila " + (f + 1));
            etiquetaFila.setFont(FUENTE_NORMAL);
            etiquetaFila.setForeground(GRIS_TEXTO);
            etiquetaFila.setPreferredSize(new Dimension(52, 30));
            fila.add(etiquetaFila);

            for (int s = 0; s < gestor.getSillasEnFila(f); s++) {
                final int numFila = f + 1;
                final int numSilla = s + 1;
                Alumno ocupante = gestor.getMatriz()[f][s];
                BotonSilla boton = new BotonSilla("F" + numFila + "-S" + numSilla,
                        ocupante == null ? "" : ocupante.getApellidoCorto());
                if (ocupante != null) {
                    boton.pintar(VERDE, Color.WHITE);
                    boton.setEnabled(false);
                }
                boton.addActionListener(e -> {
                    txtFila.setText(String.valueOf(numFila));
                    txtLugar.setText(String.valueOf(numSilla));
                });
                botonesSillas[f][s] = boton;
                fila.add(boton);
            }
            panelAula.add(fila);
            panelAula.add(Box.createVerticalStrut(6));
        }
        panelAula.revalidate();
        panelAula.repaint();
        actualizarContadores();
    }

    /** Refresca la tarjeta del alumno en turno y los contadores. */
    private void actualizarTarjeta() {
```

```
573         Alumno actual = gestor.siguienteAlumno();
574         int total = gestor.getTotalAlumnos();
575         int procesados = gestor.getProcesados();
576 
577         if (actual != null) {
578             lblTurnoNumero.setText("Alumno " + (procesados + 1) + " de " + total);
579             lblNombreAlumno.setText("<html><b>Alumno en turno:</b><br>" + actual.getNombre() + "</html>");
580             btnAceptar.setEnabled(true);
581             btnAusente.setEnabled(true);
582         } else {
583             lblTurnoNumero.setText("Alumnos procesados: " + total + " de " + total);
584             lblNombreAlumno.setText("<html><b>Cola vacia.</b><br>"
585                     + "Ya se procesaron todos los alumnos.</html>");
586             btnAceptar.setEnabled(false);
587             btnAusente.setEnabled(false);
588         }
589         txtFila.setText("");
590         txtLugar.setText("");
591         btnVerAusentes.setText("Ver lista de ausentes (" + gestor.getAusentes().size() + ")");
592         actualizarContadores();
593         actualizarPieInicio();
594     }
595 
596     private void actualizarContadores() {
597         lblContadorOcupadas.setText("Ocupadas: " + gestor.getOcupadas());
598         lblContadorLibres.setText("Libres: " + gestor.getLibres());
599         lblContadorCola.setText("En cola: " + gestor.getPendientes());
600         lblEstadoBarra.setText("Ausentes: " + gestor.getAusentes().size()
601                 + "   |   Sillas: " + gestor.getTotalSillas());
602     }
603 
604     private void asignarAsiento() {
605         try {
606             int fila = Integer.parseInt(txtFila.getText().trim());
607             int silla = Integer.parseInt(txtLugar.getText().trim());
608             Alumno asignado = gestor.asignarAsiento(fila, silla);
609 
610             actualizarBoton(fila, silla, asignado);
611             destello();
612             actualizarTarjeta();
613         } catch (NumberFormatException ex) {
614             aviso("Escriba numeros enteros en Fila y Silla.", "Dato invalido",
615                     JOptionPane.ERROR_MESSAGE);
616         } catch (GestorAula.ReglaNegocioException ex) {
617             aviso(ex.getMessage(), "No se pudo asignar la silla", JOptionPane.WARNING_MESSAGE);
618         }
619     }
620 
621     private void marcarAusente() {
622         try {
623             Alumno ausente = gestor.marcarAusente();
624             actualizarTarjeta();
```

```
            lblEstadoBarra.setText("Se marco como ausente a " + ausente.getNombre());
        } catch (GestorAula.ReglaNegocioException ex) {
            aviso(ex.getMessage(), "Cola vacia", JOptionPane.WARNING_MESSAGE);
        }
    }

    private void actualizarBoton(int fila, int silla, Alumno alumno) {
        BotonSilla boton = botonesSillas[fila - 1][silla - 1];
        if (boton != null) {
            boton.setTexto("F" + fila + "-S" + silla, alumno.getApellidoCorto());
            boton.pintar(VERDE, Color.WHITE);
            boton.setEnabled(false);
        }
    }

    /** Pequeno destello verde en la tarjeta al pasar al siguiente alumno. */
    private void destello() {
        Color original = panelTarjeta.getBackground();
        panelTarjeta.setBackground(new Color(0xE7F6EC));
        javax.swing.Timer timer = new javax.swing.Timer(180, e -> {
            panelTarjeta.setBackground(original);
            ((javax.swing.Timer) e.getSource()).stop();
        });
        timer.setRepeats(false);
        timer.start();
    }

    private void abrirAusentes() {
        JDialog dialog = new JDialog(this, "Alumnos ausentes - Dia " + diaEnCurso, true);
        dialog.setSize(560, 460);
        dialog.setLocationRelativeTo(this);

        JPanel principal = new JPanel(new BorderLayout(10, 10));
        principal.setBackground(TARJETA);
        principal.setBorder(new EmptyBorder(14, 14, 14, 14));

        JLabel titulo = new JLabel("Puede asignar una silla a un alumno que llego tarde");
        titulo.setFont(FUENTE_SECCION);
        titulo.setForeground(AZUL);
        principal.add(titulo, BorderLayout.NORTH);

        javax.swing.DefaultListModel<Alumno> modelo = new javax.swing.DefaultListModel<>();
        for (Alumno a : gestor.getAusentes()) {
            modelo.addElement(a);
        }
        JList<Alumno> lista = new JList<>(modelo);
        lista.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        lista.setFont(FUENTE_NORMAL);
        principal.add(new JScrollPane(lista), BorderLayout.CENTER);

        JPanel pie = new JPanel(new BorderLayout(8, 8));
        pie.setBackground(TARJETA);
```

```
        JPanel campos = new JPanel(new GridLayout(1, 4, 6, 0));
        campos.setBackground(TARJETA);
        JTextField fila = new JTextField();
        JTextField silla = new JTextField();
        campos.add(etiqueta("Fila:"));
        campos.add(fila);
        campos.add(etiqueta("Silla:"));
        campos.add(silla);

        Boton btnAsignar = new Boton("Asignar silla", VERDE);
        btnAsignar.setPreferredSize(new Dimension(150, 38));
        btnAsignar.addActionListener(e -> {
            Alumno seleccionado = lista.getSelectedValue();
            try {
                int f = Integer.parseInt(fila.getText().trim());
                int s = Integer.parseInt(silla.getText().trim());
                gestor.reasignarAusente(seleccionado, f, s);
                modelo.removeElement(seleccionado);
                actualizarBoton(f, s, seleccionado);
                actualizarTarjeta();
                dialog.dispose();
            } catch (NumberFormatException ex) {
                aviso("Escriba numeros enteros en Fila y Silla.", "Dato invalido",
                        JOptionPane.ERROR_MESSAGE);
            } catch (GestorAula.ReglaNegocioException ex) {
                aviso(ex.getMessage(), "No se pudo asignar la silla", JOptionPane.WARNING_MESSAGE);
            }
        });

        pie.add(campos, BorderLayout.CENTER);
        pie.add(btnAsignar, BorderLayout.EAST);
        principal.add(pie, BorderLayout.SOUTH);

        dialog.add(principal);
        dialog.setVisible(true);
    }

    // ======================================================================
    // 3. PANTALLA DE RESUMEN DEL DIA
    // ======================================================================

    private JPanel crearPantallaResumen() {
        JPanel pantalla = new JPanel(new BorderLayout());
        pantalla.setBackground(FONDO);
        pantalla.add(crearEncabezado(), BorderLayout.NORTH);

        panelResumen = new JPanel(new BorderLayout());
        panelResumen.setBackground(FONDO);
        panelResumen.setBorder(new EmptyBorder(16, 20, 16, 20));
        pantalla.add(panelResumen, BorderLayout.CENTER);
        return pantalla;
    }
```

```
729     private void mostrarResumen(RegistroDia registro) {
730         panelResumen.removeAll();
731 
732         JPanel superior = new JPanel(new BorderLayout(0, 12));
733         superior.setBackground(FONDO);
734 
735         JLabel titulo = new JLabel("Jornada finalizada: " + registro.getTitulo());
736         titulo.setFont(FUENTE_TITULO);
737         titulo.setForeground(AZUL);
738         superior.add(titulo, BorderLayout.NORTH);
739 
740         JPanel kpis = new JPanel(new GridLayout(1, 4, 12, 0));
741         kpis.setBackground(FONDO);
742         kpis.add(tarjetaKpi(String.valueOf(registro.getTotalAlumnos()), "Alumnos en lista", AZUL));
743         kpis.add(tarjetaKpi(String.valueOf(registro.getAsistentes()), "Asistieron", VERDE));
744         kpis.add(tarjetaKpi(String.valueOf(registro.getFaltas()), "Faltas", ROJO));
745         kpis.add(tarjetaKpi(registro.getPorcentajeAsistencia() + "%", "Asistencia", AZUL_MEDIO));
746         superior.add(kpis, BorderLayout.CENTER);
747         panelResumen.add(superior, BorderLayout.NORTH);
748 
749         JPanel aula = new JPanel();
750         aula.setBackground(FONDO);
751         aula.setLayout(new BoxLayout(aula, BoxLayout.Y_AXIS));
752         aula.setBorder(new EmptyBorder(16, 0, 0, 0));
753         dibujarAulaRegistro(aula, registro);
754 
755         JScrollPane scroll = new JScrollPane(aula);
756         scroll.setBorder(null);
757         scroll.getVerticalScrollBar().setUnitIncrement(16);
758         scroll.setBackground(FONDO);
759         panelResumen.add(scroll, BorderLayout.CENTER);
760 
761         JPanel botones = new JPanel(new BorderLayout());
762         botones.setBackground(FONDO);
763         botones.setBorder(new EmptyBorder(12, 0, 0, 0));
764 
765         Boton btnComparar = new Boton("Ver comparativa de los 2 dias", AZUL_MEDIO);
766         btnComparar.addActionListener(e -> mostrarComparativa());
767         btnComparar.setEnabled(gestor.getHistorial().size() == GestorAula.TOTAL_DIAS);
768 
769         Boton btnInicio = new Boton("Regresar al inicio", VERDE);
770         btnInicio.addActionListener(e -> mostrarInicio());
771 
772         botones.add(btnComparar, BorderLayout.WEST);
773         botones.add(btnInicio, BorderLayout.EAST);
774         panelResumen.add(botones, BorderLayout.SOUTH);
775 
776         panelResumen.revalidate();
777         panelResumen.repaint();
778     }
```

```
781     /** Pinta el aula de un dia ya guardado (sin interaccion). */
782     private void dibujarAulaRegistro(JPanel contenedor, RegistroDia registro) {
783         Alumno[][] matriz = new Alumno[registro.getFilas()][];
784         for (int f = 0; f < registro.getFilas(); f++) {
785             matriz[f] = new Alumno[registro.getSillasPorFila(f)];
786         }
787         for (Alumno a : registro.getAlumnos()) {
788             if (a.tieneAsiento() && a.isAsistio()) {
789                 matriz[a.getFila() - 1][a.getLugar() - 1] = a;
790             }
791         }
792 
793         for (int f = 0; f < registro.getFilas(); f++) {
794             JPanel fila = new JPanel(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT, 8, 6));
795             fila.setBackground(FONDO);
796             fila.setAlignmentX(Component.LEFT_ALIGNMENT);
797             fila.setMaximumSize(new Dimension(Integer.MAX_VALUE, 74));
798 
799             JLabel etiquetaFila = new JLabel("Fila " + (f + 1));
800             etiquetaFila.setFont(FUENTE_NORMAL);
801             etiquetaFila.setForeground(GRIS_TEXTO);
802             etiquetaFila.setPreferredSize(new Dimension(52, 30));
803             fila.add(etiquetaFila);
804 
805             for (int s = 0; s < registro.getSillasPorFila(f); s++) {
806                 Alumno ocupante = matriz[f][s];
807                 BotonSilla boton = new BotonSilla("F" + (f + 1) + "-S" + (s + 1),
808                         ocupante == null ? "" : ocupante.getApellidoCorto());
809                 boton.setEnabled(false);
810                 if (ocupante != null) {
811                     boton.pintar(VERDE, Color.WHITE);
812                 }
813                 fila.add(boton);
814             }
815             contenedor.add(fila);
816             contenedor.add(Box.createVerticalStrut(6));
817         }
818     }
819 
820     private JPanel tarjetaKpi(String valor, String texto, Color color) {
821         JPanel tarjeta = tarjetaBlanca();
822         tarjeta.setLayout(new BoxLayout(tarjeta, BoxLayout.Y_AXIS));
823         tarjeta.setBorder(new EmptyBorder(14, 18, 14, 18));
824 
825         JLabel lblValor = new JLabel(valor);
826         lblValor.setFont(new Font("Segoe UI", Font.BOLD, 32));
827         lblValor.setForeground(color);
828         lblValor.setAlignmentX(Component.LEFT_ALIGNMENT);
829 
830         JLabel lblTexto = new JLabel(texto);
831         lblTexto.setFont(FUENTE_NORMAL);
832         lblTexto.setForeground(GRIS_TEXTO);
```

```
833         lblTexto.setAlignmentX(Component.LEFT_ALIGNMENT);
834
835         tarjeta.add(lblValor);
836         tarjeta.add(lblTexto);
837         return tarjeta;
838     }
839
840     // ========================================================================
841     // 4. PANTALLA COMPARATIVA DE LOS 2 DIAS
842     // ========================================================================
843
844     private JPanel crearPantallaComparativa() {
845         JPanel pantalla = new JPanel(new BorderLayout());
846         pantalla.setBackground(FONDO);
847         pantalla.add(crearEncabezado(), BorderLayout.NORTH);
848
849         panelComparativa = new JPanel(new BorderLayout(0, 14));
850         panelComparativa.setBackground(FONDO);
851         panelComparativa.setBorder(new EmptyBorder(16, 20, 16, 20));
852         pantalla.add(panelComparativa, BorderLayout.CENTER);
853         return pantalla;
854     }
855
856     private void mostrarComparativa() {
857         panelComparativa.removeAll();
858         RegistroDia dia1 = gestor.buscarDia(1);
859         RegistroDia dia2 = gestor.buscarDia(2);
860
861         JLabel titulo = new JLabel("Comparativa de las 2 jornadas");
862         titulo.setFont(FUENTE_TITULO);
863         titulo.setForeground(AZUL);
864         panelComparativa.add(titulo, BorderLayout.NORTH);
865
866         JPanel centro = new JPanel(new BorderLayout(0, 14));
867         centro.setBackground(FONDO);
868
869         // Tarjetas resumen por dia
870         JPanel filaKpis = new JPanel(new GridLayout(1, 2, 14, 0));
871         filaKpis.setBackground(FONDO);
872         filaKpis.add(panelDia(dia1));
873         filaKpis.add(panelDia(dia2));
874         centro.add(filaKpis, BorderLayout.NORTH);
875
876         // Tabla alumno por alumno
877         String[] columnas = {"Alumno", "Dia 1: asiento", "Dia 1: asistencia",
878                 "Dia 2: asiento", "Dia 2: asistencia"};
879         DefaultTableModel modelo = new DefaultTableModel(columnas, 0) {
880             @Override
881             public boolean isCellEditable(int fila, int columna) {
882                 return false;
883             }
884         };
```

```
885         List<Alumno> base = dia1 != null ? dia1.getAlumnos() : new ArrayList<>();
886         for (Alumno alumno : base) {
887             Alumno enDia2 = dia2 != null ? dia2.buscar(alumno.getNombre()) : null;
888             modelo.addRow(new Object[]{
889                     alumno.getNombre(),
890                     textoAsiento(alumno),
891                     alumno.isAsistio() ? "Asistio" : "Falta",
892                     enDia2 == null ? "-" : textoAsiento(enDia2),
893                     enDia2 == null ? "-" : (enDia2.isAsistio() ? "Asistio" : "Falta")
894             });
895         }
896 
897         JTable tabla = new JTable(modelo);
898         tabla.setFont(FUENTE_NORMAL);
899         tabla.setRowHeight(28);
900         tabla.getTableHeader().setFont(new Font("Segoe UI", Font.BOLD, 13));
901         tabla.setGridColor(new Color(0xE2E7EE));
902         DefaultTableCellRenderer centrado = new DefaultTableCellRenderer();
903         centrado.setHorizontalAlignment(SwingConstants.CENTER);
904         for (int c = 1; c < tabla.getColumnCount(); c++) {
905             tabla.getColumnModel().getColumn(c).setCellRenderer(centrado);
906         }
907 
908         JScrollPane scroll = new JScrollPane(tabla);
909         scroll.setBorder(BorderFactory.createTitledBorder("Asistencia y asiento por dia"));
910         centro.add(scroll, BorderLayout.CENTER);
911         panelComparativa.add(centro, BorderLayout.CENTER);
912 
913         Boton btnInicio = new Boton("Regresar al inicio", VERDE);
914         btnInicio.addActionListener(e -> mostrarInicio());
915         JPanel pie = new JPanel(new BorderLayout());
916         pie.setBackground(FONDO);
917         pie.add(btnInicio, BorderLayout.EAST);
918         panelComparativa.add(pie, BorderLayout.SOUTH);
919 
920         panelComparativa.revalidate();
921         panelComparativa.repaint();
922         cardLayout.show(contenedor, "comparativa");
923     }
924 
925     private JPanel panelDia(RegistroDia dia) {
926         JPanel panel = tarjetaBlanca();
927         panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
928         panel.setBorder(new EmptyBorder(14, 18, 14, 18));
929 
930         JLabel titulo = new JLabel(dia == null ? "Dia pendiente" : dia.getTitulo());
931         titulo.setFont(FUENTE_SECCION);
932         titulo.setForeground(dia == null ? GRIS_TEXTO : AZUL);
933         titulo.setAlignmentX(Component.LEFT_ALIGNMENT);
934         panel.add(titulo);
```

```
937         if (dia == null) {
938             JLabel pendiente = new JLabel("Aun no se ha registrado esta jornada.");
939             pendiente.setFont(FUENTE_NORMAL);
940             pendiente.setForeground(GRIS_TEXTO);
941             pendiente.setAlignmentX(Component.LEFT_ALIGNMENT);
942             panel.add(pendiente);
943             return panel;
944         }
945 
946         panel.add(Box.createVerticalStrut(6));
947         panel.add(linea("Asistieron: " + dia.getAsistentes() + " de " + dia.getTotalAlumnos()
948                 + " (" + dia.getPorcentajeAsistencia() + "%)", VERDE));
949         panel.add(linea("Faltas: " + dia.getFaltas(), ROJO));
950         panel.add(linea("Sillas ocupadas: " + dia.getSillasOcupadas() + " de " + dia.getTotalSillas(), AZUL_MEDIO));
951         return panel;
952     }
953 
954     private JLabel linea(String texto, Color color) {
955         JLabel lbl = new JLabel(texto);
956         lbl.setFont(FUENTE_NORMAL);
957         lbl.setForeground(color);
958         lbl.setAlignmentX(Component.LEFT_ALIGNMENT);
959         return lbl;
960     }
961 
962     private String textoAsiento(Alumno alumno) {
963         return alumno.tieneAsiento() ? "F" + alumno.getFila() + "-S" + alumno.getLugar() : "-";
964     }
965 
966     // ========================================================================
967     // Navegacion y utilidades de interfaz
968     // ========================================================================
969 
970     private void finalizarJornada() {
971         int respuesta = JOptionPane.showConfirmDialog(this,
972                 "Finalizar la jornada del " + diaSemanaEnCurso + "?\n"
973                         + "Los alumnos que sigan en la cola se contaran como FALTAS.",
974                 "Confirmar cierre del dia", JOptionPane.YES_NO_OPTION);
975         if (respuesta != JOptionPane.YES_OPTION) {
976             return;
977         }
978 
979         RegistroDia registro = gestor.finalizarDia(diaEnCurso, diaSemanaEnCurso);
980         try {
981             gestor.guardarHistorial(archivoHistorial);
982         } catch (IOException ex) {
983             aviso("El resumen se mostrara, pero no se pudo guardar el historial: "
984                     + ex.getMessage(), "Aviso", JOptionPane.WARNING_MESSAGE);
985         }
986         mostrarResumen(registro);
987         cardLayout.show(contenedor, "resumen");
988     }
```

```
    private void mostrarInicio() {
        actualizarPieInicio();
        String[] opciones = {"Dia 1", "Dia 2"};
        for (int i = 0; i < opciones.length; i++) {
            if (gestor.diaRegistrado(i + 1)) {
                opciones[i] = opciones[i] + " (registrado)";
            }
        }
        cbJornada.setModel(new javax.swing.DefaultComboBoxModel<>(opciones));
        cbJornada.setSelectedIndex(gestor.diaRegistrado(1) ? 1 : 0);
        cardLayout.show(contenedor, "inicio");
    }

    private void cargarHistorialGuardado() {
        try {
            gestor.cargarHistorial(archivoHistorial);
        } catch (IOException | RuntimeException ex) {
            // Si el archivo esta danado se ignora: la app debe abrir igual.
            System.err.println("No se pudo cargar el historial: " + ex.getMessage());
        }
    }

    private void aviso(String mensaje, String titulo, int tipo) {
        JOptionPane.showMessageDialog(this, mensaje, titulo, tipo);
    }

    private JPanel crearEncabezado() {
        JPanel encabezado = new JPanel(new BorderLayout()) {
            @Override
            protected void paintComponent(Graphics g) {
                Graphics2D g2 = (Graphics2D) g.create();
                g2.setPaint(new GradientPaint(0, 0, AZUL, getWidth(), 0, new Color(0x2A4C77)));
                g2.fillRect(0, 0, getWidth(), getHeight());
                g2.dispose();
            }
        };
        encabezado.setBorder(new EmptyBorder(14, 22, 14, 22));

        JLabel titulo = new JLabel("Gestion de Asientos en el Aula");
        titulo.setFont(new Font("Segoe UI", Font.BOLD, 21));
        titulo.setForeground(Color.WHITE);

        JLabel subtitulo = new JLabel("Estructura de datos lineal: COLA (FIFO)  |  Unidad 2 - Caso practico");
        subtitulo.setFont(FUENTE_SUBTITULO);
        subtitulo.setForeground(new Color(0xC9D8EA));

        JPanel textos = new JPanel();
        textos.setOpaque(false);
        textos.setLayout(new BoxLayout(textos, BoxLayout.Y_AXIS));
        textos.add(titulo);
        textos.add(subtitulo);
```

```
1041        encabezado.add(textos, BorderLayout.WEST);
1042
1043        Boton btnHistorial = new Boton("Ver comparativa", new Color(0x3C6E9B));
1044        btnHistorial.setPreferredSize(new Dimension(150, 36));
1045        btnHistorial.addActionListener(e -> {
1046            if (gestor.getHistorial().isEmpty()) {
1047                aviso("Todavia no hay jornadas registradas.", "Sin historial",
1048                        JOptionPane.INFORMATION_MESSAGE);
1049                return;
1050            }
1051            mostrarComparativa();
1052        });
1053        JPanel derecha = new JPanel(new java.awt.FlowLayout(java.awt.FlowLayout.RIGHT, 0, 0));
1054        derecha.setOpaque(false);
1055        derecha.add(btnHistorial);
1056        encabezado.add(derecha, BorderLayout.EAST);
1057        return encabezado;
1058    }
1059
1060    private JPanel tarjetaBlanca() {
1061        JPanel panel = new JPanel();
1062        panel.setBackground(TARJETA);
1063        panel.setBorder(BorderFactory.createCompoundBorder(
1064                BorderFactory.createLineBorder(new Color(0xE2E7EE)),
1065                new EmptyBorder(2, 2, 2, 2)));
1066        return panel;
1067    }
1068
1069    private JLabel etiqueta(String texto) {
1070        JLabel lbl = new JLabel(texto);
1071        lbl.setFont(FUENTE_NORMAL);
1072        lbl.setForeground(GRIS_TEXTO);
1073        return lbl;
1074    }
1075
1076    /** Etiqueta de ancho fijo para alinear los formularios. */
1077    private JLabel etiquetaAncha(String texto) {
1078        JLabel lbl = etiqueta(texto);
1079        lbl.setPreferredSize(new Dimension(190, 24));
1080        return lbl;
1081    }
1082
1083    private JLabel contador(String texto) {
1084        JLabel lbl = new JLabel(texto);
1085        lbl.setFont(new Font("Segoe UI", Font.BOLD, 13));
1086        lbl.setForeground(AZUL);
1087        return lbl;
1088    }
1089
1090    /** Boton plano con color propio y efecto de hover. */
1091    private static class Boton extends JButton {
1092        private final Color color;
```

```
        private boolean encima = false;

        Boton(String texto, Color color) {
            super(texto);
            this.color = color;
            setFont(FUENTE_BOTON);
            setForeground(Color.WHITE);
            setContentAreaFilled(false);
            setFocusPainted(false);
            setBorderPainted(false);
            setOpaque(false);
            setCursor(new Cursor(Cursor.HAND_CURSOR));
            setPreferredSize(new Dimension(200, 38));
            addMouseListener(new java.awt.event.MouseAdapter() {
                @Override
                public void mouseEntered(java.awt.event.MouseEvent e) {
                    encima = true;
                    repaint();
                }

                @Override
                public void mouseExited(java.awt.event.MouseEvent e) {
                    encima = false;
                    repaint();
                }
            });
        }

        @Override
        protected void paintComponent(Graphics g) {
            Graphics2D g2 = (Graphics2D) g.create();
            g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
            Color base = isEnabled() ? color : new Color(0xB6BEC9);
            if (encima && isEnabled()) {
                base = base.brighter();
            }
            g2.setColor(base);
            g2.fillRoundRect(0, 0, getWidth() - 1, getHeight() - 1, 10, 10);
            g2.setColor(new Color(255, 255, 255, 60));
            g2.setStroke(new BasicStroke(1f));
            g2.drawRoundRect(0, 0, getWidth() - 1, getHeight() - 1, 10, 10);

            g2.setFont(getFont());
            g2.setColor(getForeground());
            FontMetrics fm = g2.getFontMetrics();
            int x = (getWidth() - fm.stringWidth(getText())) / 2;
            int y = (getHeight() + fm.getAscent() - fm.getDescent()) / 2;
            g2.drawString(getText(), x, y);
            g2.dispose();
        }
    }
```

```
    /** Boton de una silla: dibuja el codigo F?-S? y el apellido del ocupante. */
    private static class BotonSilla extends JButton {
        private String linea1;
        private String linea2;
        private Color fondo = new Color(0xFFFFFF);
        private Color texto = new Color(0x3A4552);

        BotonSilla(String linea1, String linea2) {
            this.linea1 = linea1;
            this.linea2 = linea2;
            Dimension tamano = new Dimension(104, 58);
            setPreferredSize(tamano);
            setMinimumSize(tamano);
            setMaximumSize(tamano);
            setContentAreaFilled(false);
            setFocusPainted(false);
            setBorderPainted(false);
            setOpaque(false);
            setCursor(new Cursor(Cursor.HAND_CURSOR));
        }

        void setTexto(String linea1, String linea2) {
            this.linea1 = linea1;
            this.linea2 = linea2;
            repaint();
        }

        void pintar(Color fondo, Color texto) {
            this.fondo = fondo;
            this.texto = texto;
            repaint();
        }

        @Override
        protected void paintComponent(Graphics g) {
            Graphics2D g2 = (Graphics2D) g.create();
            g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
            g2.setColor(fondo);
            g2.fillRoundRect(0, 0, getWidth() - 1, getHeight() - 1, 12, 12);
            g2.setColor(new Color(0, 0, 0, 26));
            g2.drawRoundRect(0, 0, getWidth() - 1, getHeight() - 1, 12, 12);

            g2.setColor(texto);
            g2.setFont(new Font("Segoe UI", Font.BOLD, 11));
            FontMetrics fm = g2.getFontMetrics();
            g2.drawString(linea1, (getWidth() - fm.stringWidth(linea1)) / 2, 22);

            if (linea2 != null && !linea2.isEmpty()) {
                g2.setFont(new Font("Segoe UI", Font.PLAIN, 10));
                fm = g2.getFontMetrics();
                g2.drawString(linea2, (getWidth() - fm.stringWidth(linea2)) / 2, 39);
            }
```

```
            g2.dispose();
        }
    }

    // ====================================================================
    // Punto de entrada
    // ====================================================================

    public static void main(String[] args) {
        try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        } catch (Exception ex) {
            System.err.println("No se pudo aplicar el look and feel: " + ex.getMessage());
        }
        SwingUtilities.invokeLater(() -> new GestionAsientosAula().setVisible(true));
    }
}
```

Pruebas de la lógica

La lógica se probó por separado, sin interfaz, con la clase PruebasGestorAula. Las pruebas cubren el orden FIFO, la silla ocupada, los rangos inválidos, el cierre del día con faltas y la persistencia de dos jornadas.

Resultado de la ejecución: 15 pruebas ejecutadas, 0 fallidas.

Ejecución

Las siguientes capturas corresponden a la ejecución real del programa. Primero se muestran los casos de éxito y después los casos de fallo solicitados en la plantilla.

Casos de éxito

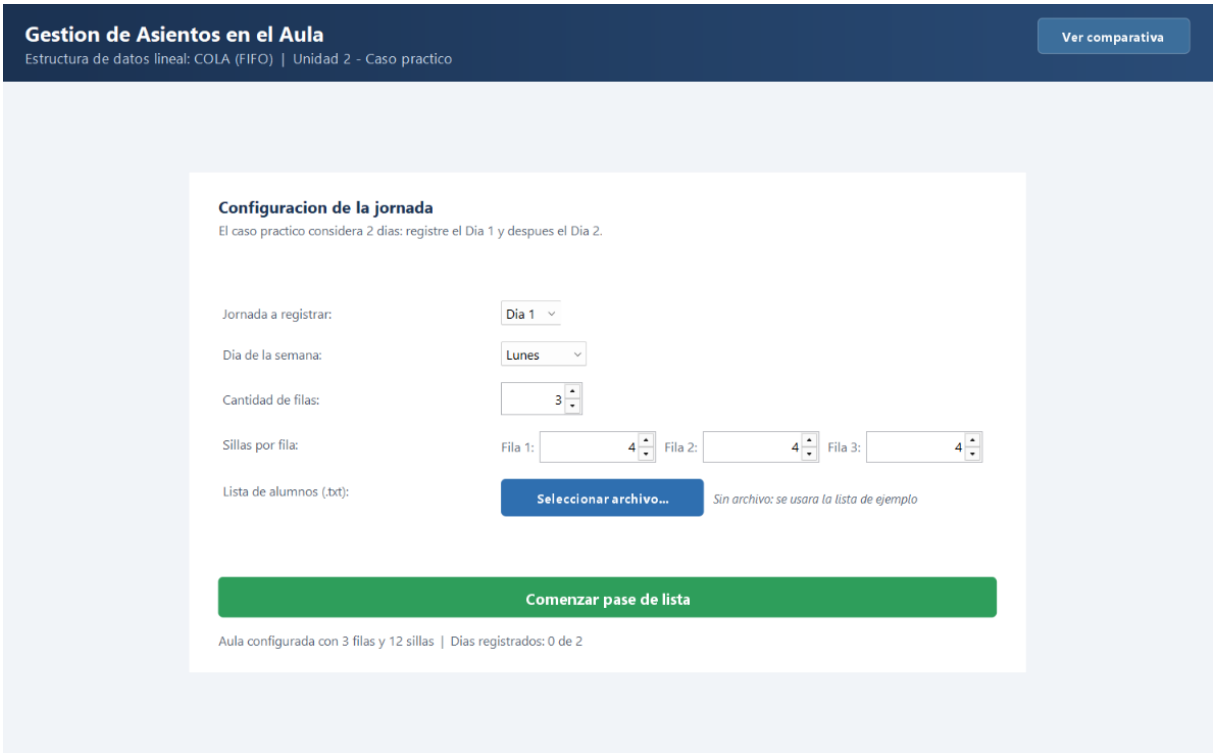


Figura 3. Configuración del Día 1: 3 filas, 4 sillas por fila y lista de alumnos.

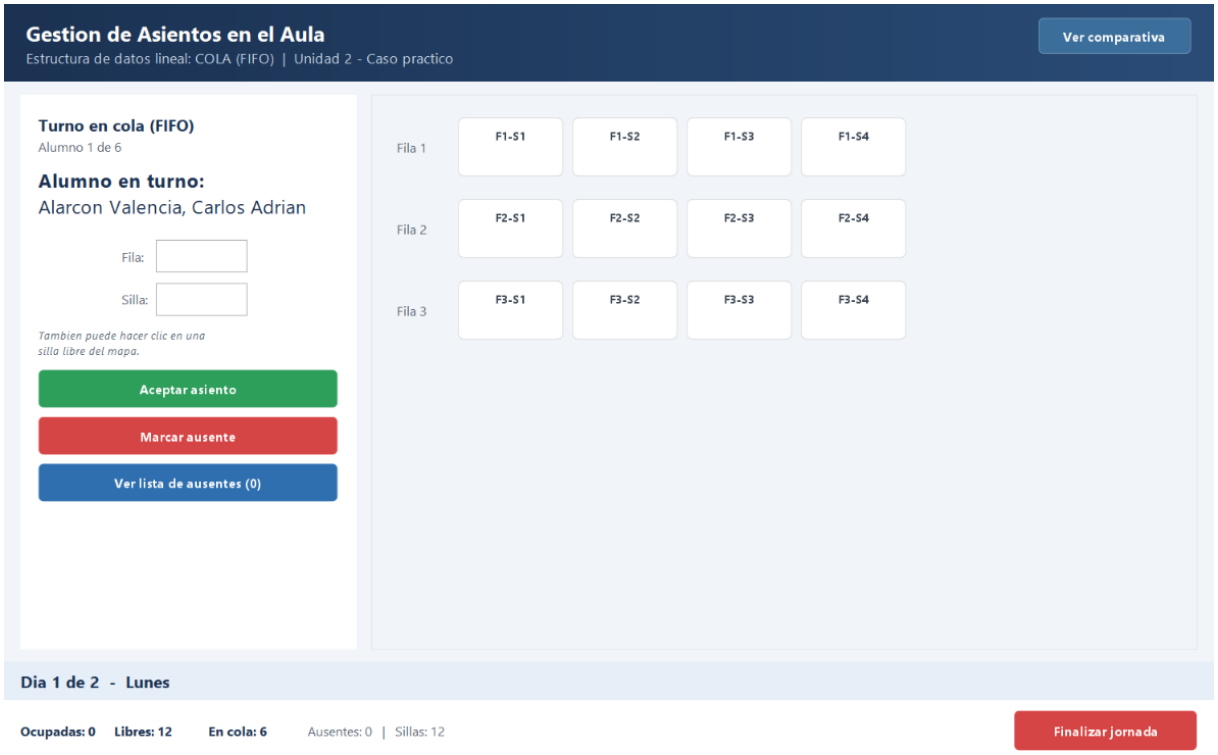


Figura 4. Inicio de la jornada: la cola tiene 6 alumnos y el aula está vacía.

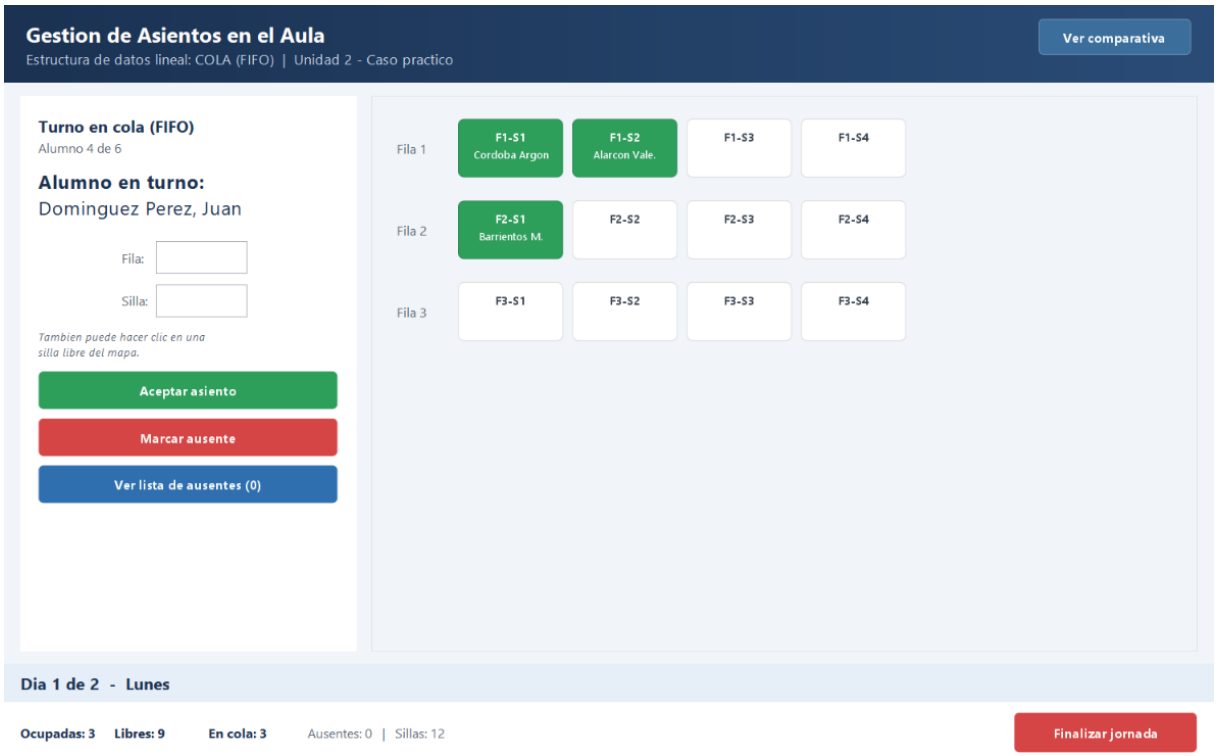


Figura 5. Tres alumnos ya eligieron su silla; el cuarto está en turno.

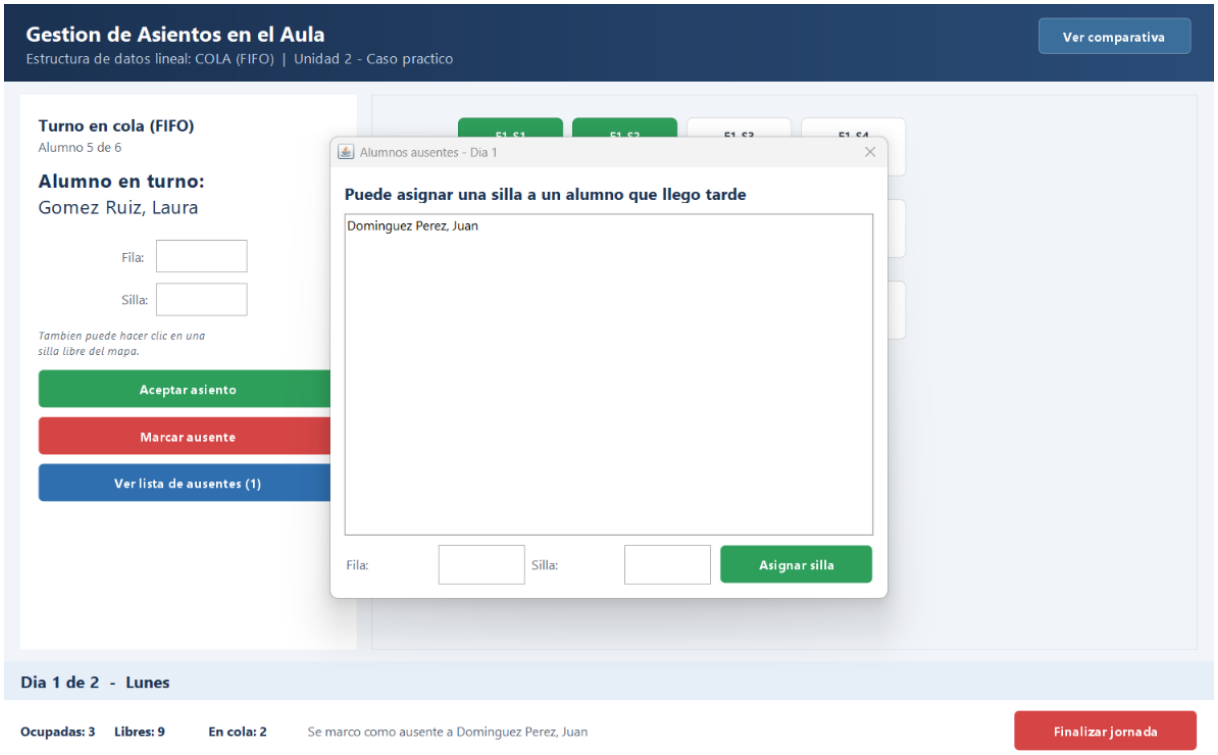


Figura 6. Alumno marcado como ausente y reasignación de una silla al llegar tarde.

Casos de fallo

El programa impide las tres situaciones incorrectas más frecuentes y conserva al alumno en la cola para que elija otra silla:

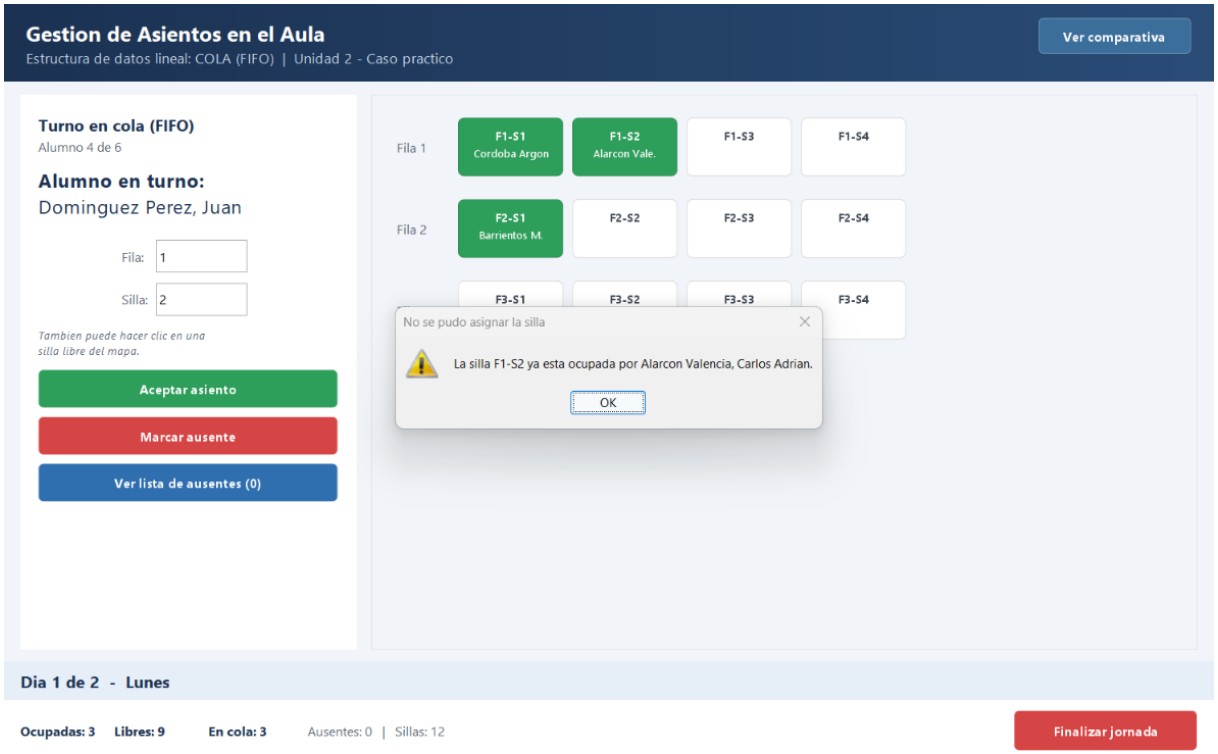


Figura 7. Silla ocupada: el sistema indica quién la ocupa y no realiza la asignación.

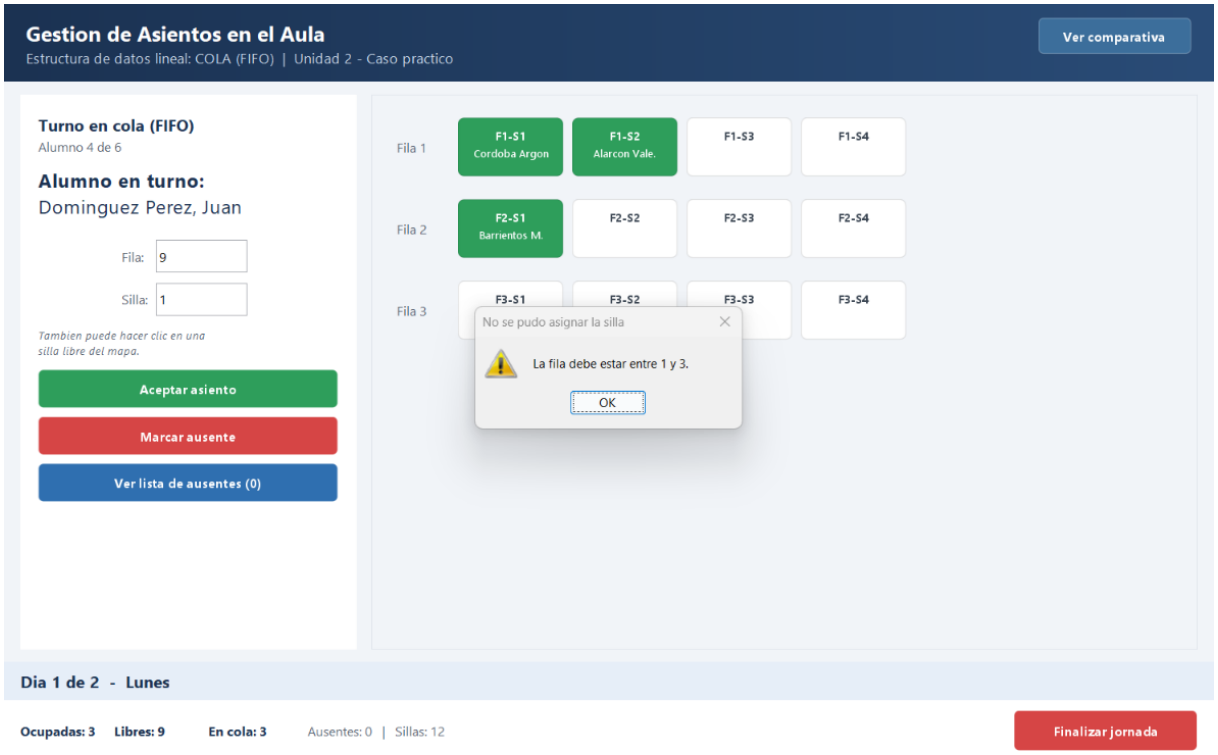


Figura 8. Fila inexistente: se valida el rango antes de modificar la matriz.

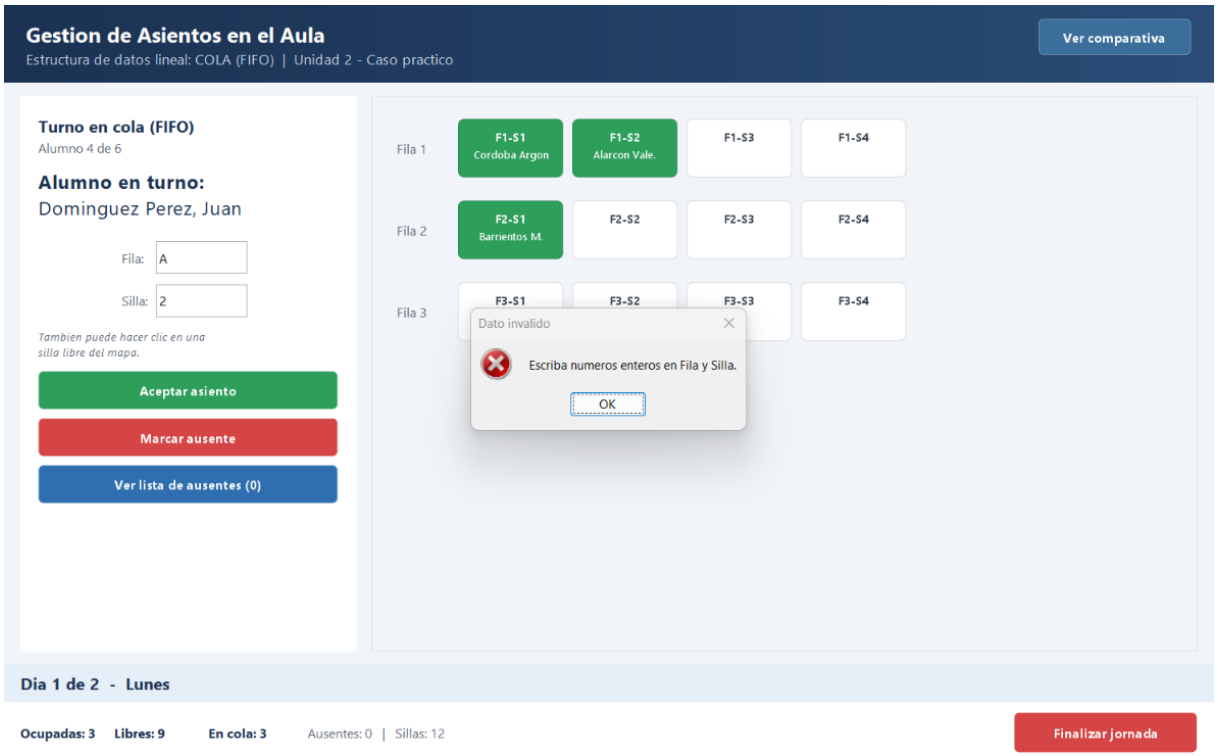


Figura 9. Dato no numérico: se solicita capturar números enteros.

Cierre del día y comparación de las dos jornadas

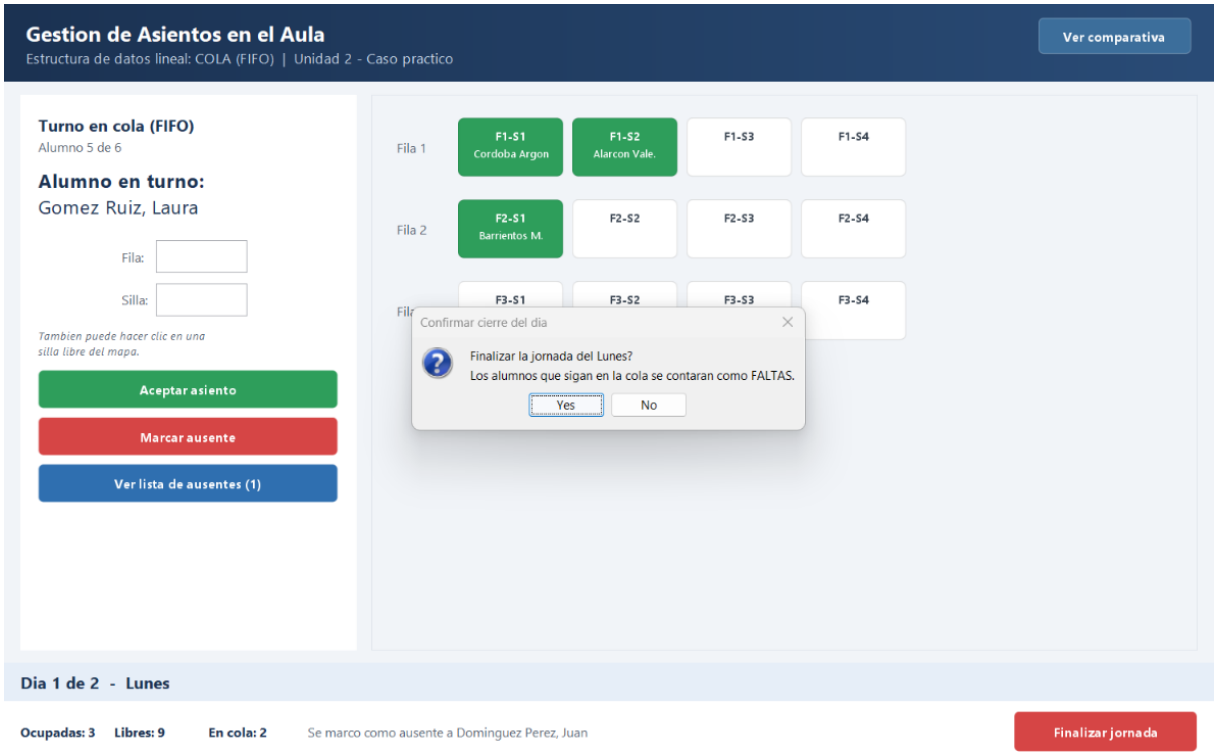


Figura 10. Confirmación del cierre: los alumnos pendientes se cuentan como faltas.

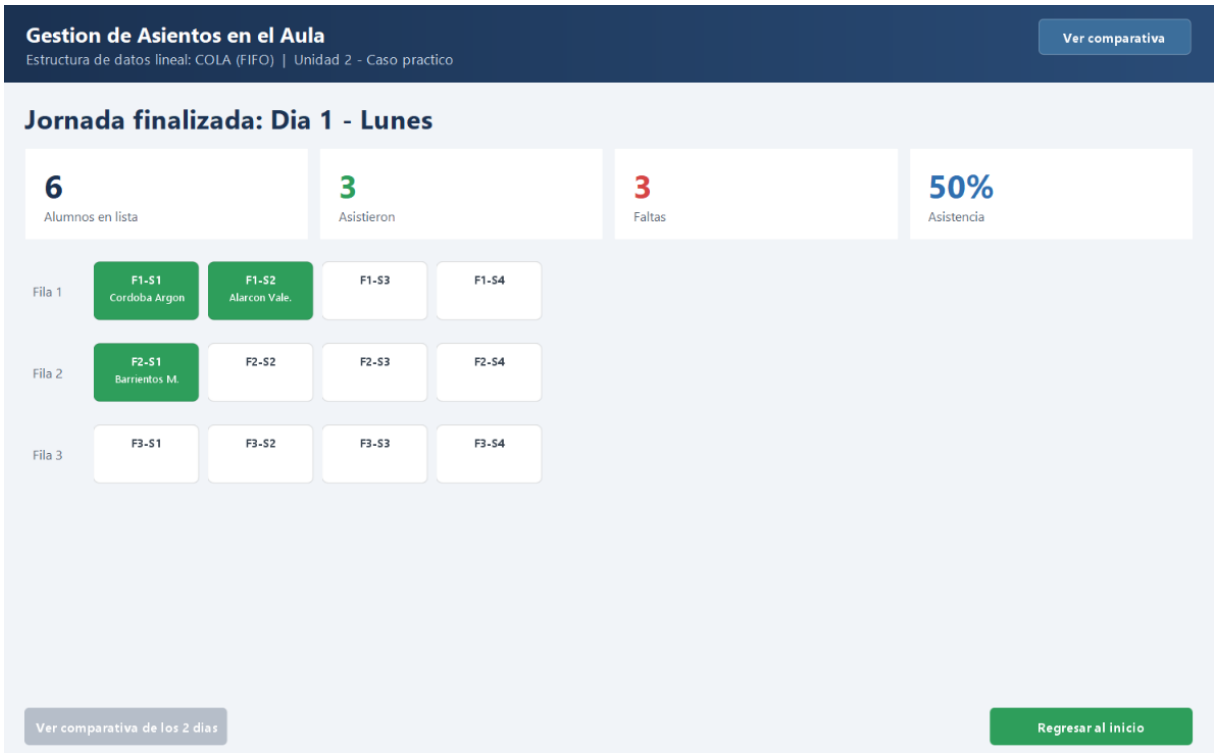


Figura 11. Resumen del Día 1 con totales y mapa de asientos en solo lectura.

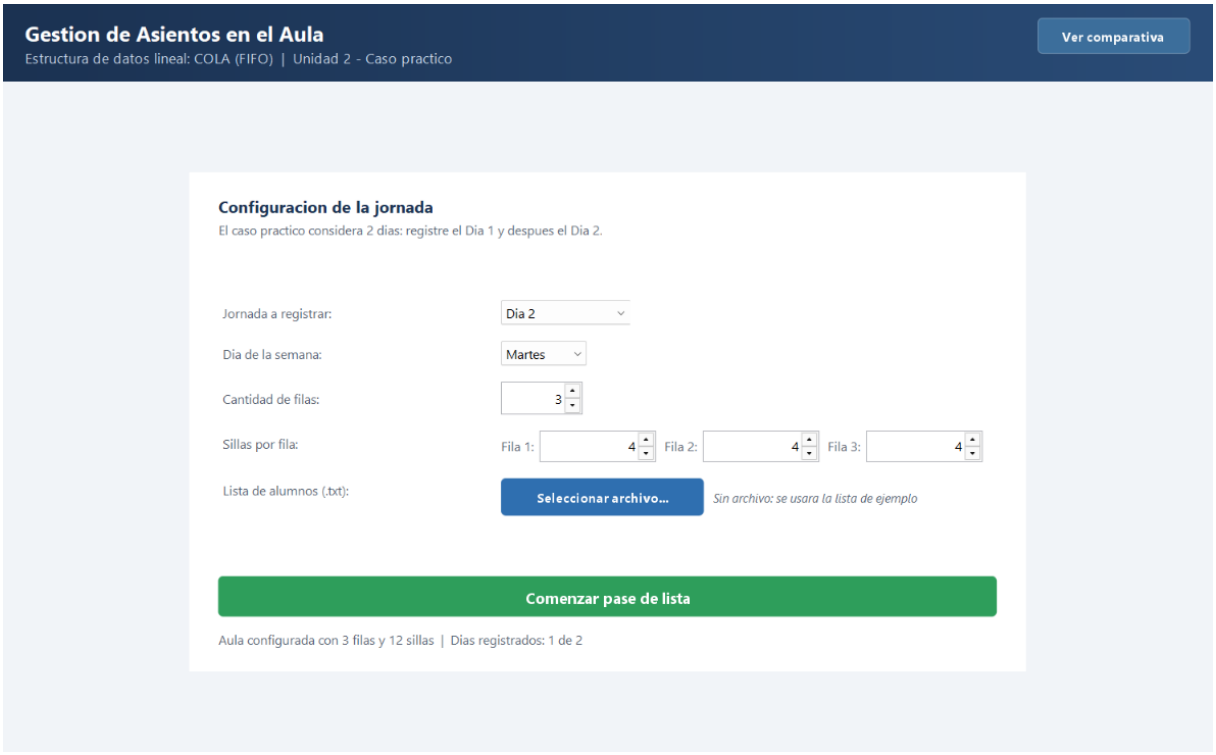


Figura 12. El Día 1 queda registrado y el sistema habilita el Día 2.

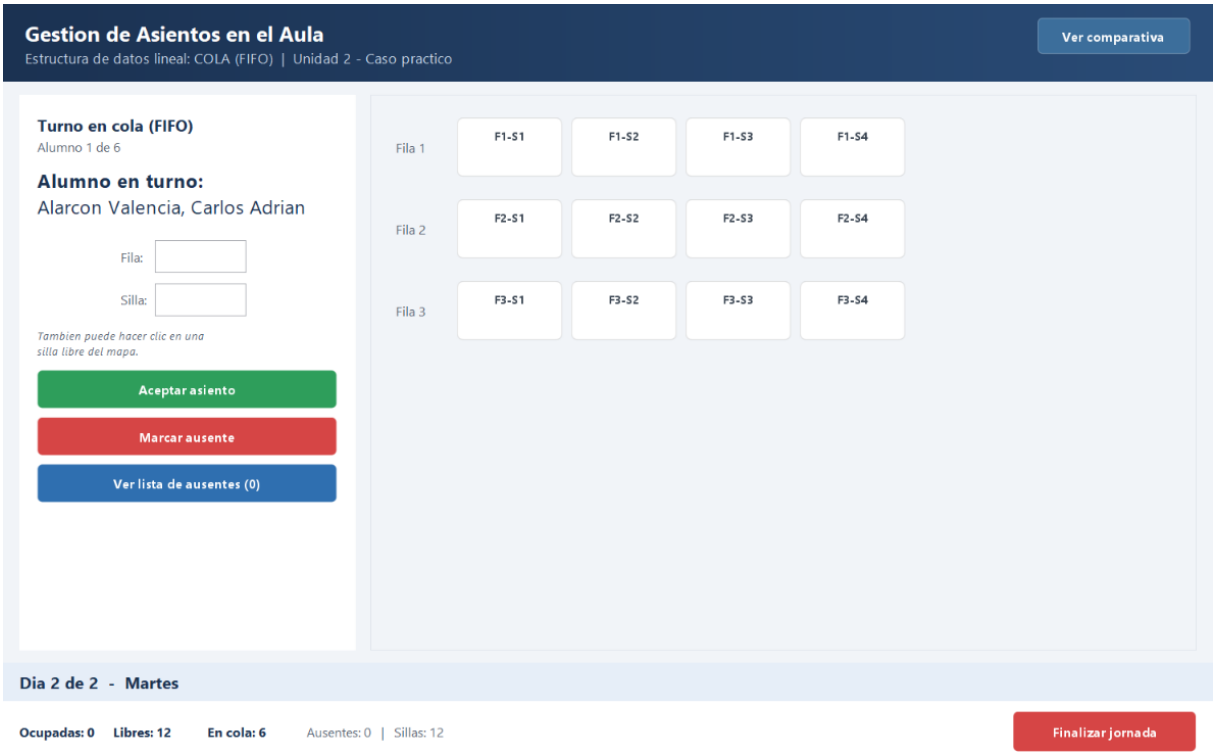


Figura 13. Día 2 en curso: el mapa de asientos se reinicia y los alumnos vuelven a elegir.

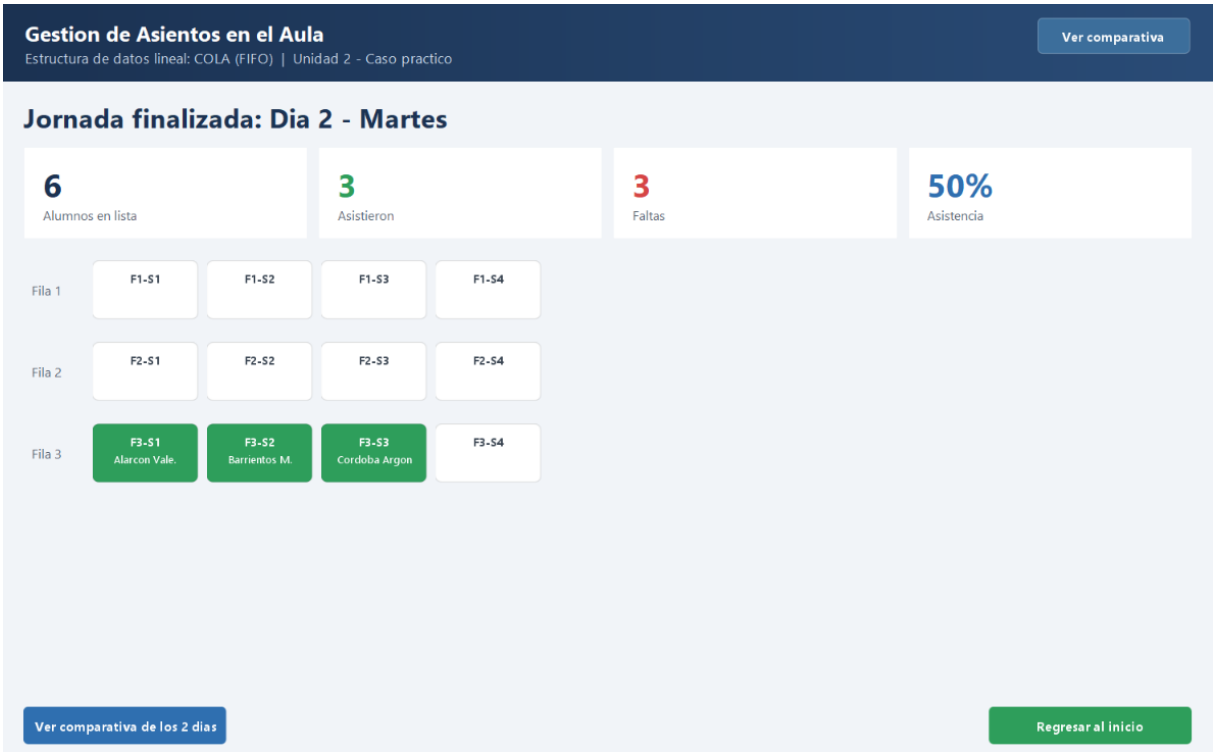


Figura 14. Resumen del Día 2.

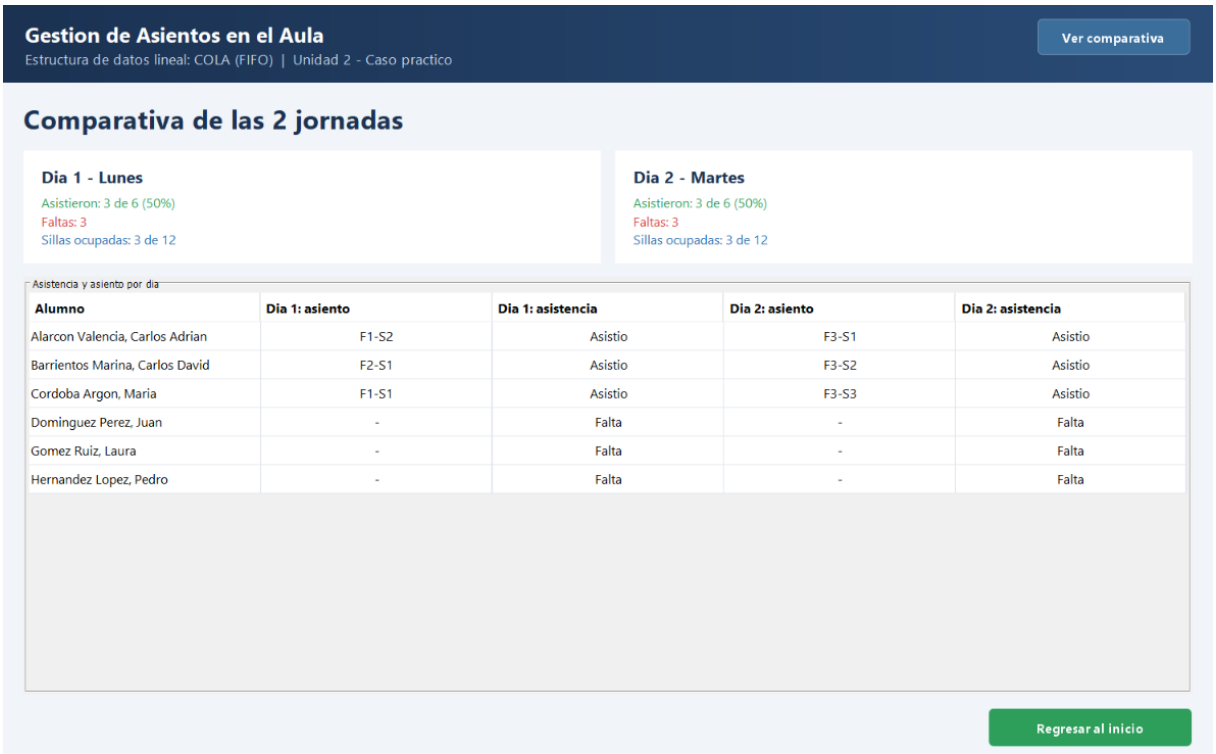


Figura 15. Comparativa de las dos jornadas: asiento y asistencia por alumno y por día.

Conclusiones

El caso se resolvió con una cola porque el pase de lista es, por definición, un procedimiento FIFO. Las operaciones offer, peek y poll permiten expresar el turno sin ordenamientos ni búsquedas adicionales, y la matriz de asientos resuelve la elección de lugar con acceso directo en tiempo constante.

El registro por día se resolvió guardando una copia del estado al cerrar cada jornada, de modo que el Día 2 no altera la información del Día 1. Con eso, el programa no solo cumple el enunciado, sino que permite comparar las dos jornadas y conservar el historial entre ejecuciones.

Finalmente, separar la lógica de la interfaz permitió probar las reglas del caso sin ventanas: las quince pruebas automatizadas verifican el orden de la cola, las validaciones de los asientos, el cierre con faltas y la persistencia de los dos días.

Anexo. Manual de usuario

1. Ejecutar el programa con java -cp build GestionAsientosAula.
2. Elegir la jornada (Día 1 o Día 2), el día de la semana, el número de filas y las sillas de cada fila. De manera opcional, cargar la lista de alumnos desde un archivo .txt con un alumno por línea, empezando por apellidos.
3. Presionar «Comenzar pase de lista».
4. Para cada alumno en turno, escribir la fila y la silla (o hacer clic sobre una silla libre) y presionar «Aceptar asiento». Si el alumno no llegó, presionar «Marcar ausente».
5. Si un alumno ausente llega tarde, abrir «Ver lista de ausentes», seleccionarlo, indicar fila y silla y presionar «Asignar silla».
6. Al terminar, presionar «Finalizar jornada» y confirmar. El resumen del día se muestra de inmediato y el historial se guarda en historial_asientos.tsv.
7. Regresar al inicio y repetir el procedimiento para el Día 2.
8. Abrir «Ver comparativa» para consultar la asistencia y el asiento de cada alumno en las dos jornadas.